INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2025

COURSE: [F25] Networks Engineering

INR Lab 2 Report

IPv4 & IPv6Assignment

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 13.11.2025

Appendix:

Questions for Lab 2 3-5

Task 1 Solutions (Part 1 – 3) 5-8

Task 2 Solutions 9-11

Task 3 Solutions 12-36

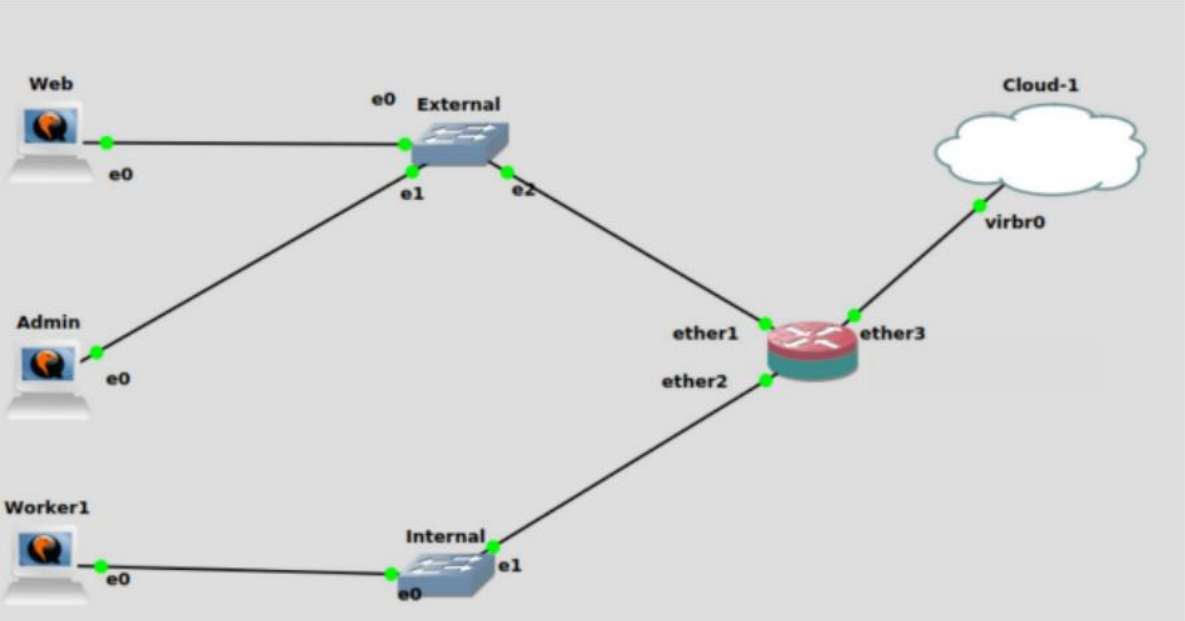
Task 1 Part 4 Solutions 37-41

References 41-43

Task 1 - Ports and Protocols

Using your network topology from the previous lab, you will be gathering some information about your machines.

You might need to substitute the "dummy" Worker VPC node from Lab 1 topology by the Ubuntu Cloud Guest instance. So, the lab topology could be as following:



1. Check the open ports and listening Unix sockets against ssh (22) and http (80) on Admin and Web respectively.

Hint: use both lsof and netstat

2. Scan your gateway from the outside. What are the known open ports?

Hint: use nmap

3. A gateway has to be transparent, you should not see any port that is not specifically forwarded. Adjust your firewall rules to make this happen. Disable any unnecessary services and scan again.

4. It suppose that some scanners start by scanning the known ports and pinging a host to see if it is alive.

4.1. Scan the Worker VM from Admin. Can you see any ports?

4.2. Block ICMP traffic on Worker and change the port for SSH to one that is above 10000.

4.3. Scan it without extra arguments.

4.4. Now make necessary changes to the command to force the scan on all possible ports.

Note. Nmap doesn't only use ping for discovery but also other probes, you can read about this the documentation, Chapter 3: Host Discovery.

4.5. Gather some information about your open ports on Web (ssh and http).

Note. Don't paste the scan results, summarize them in the answer and include them as an appendix of your submission in Moodle.

Task 2 - Traffic Captures

In some cases, you might need to take a look at the traffic sent and received from your machines to understand what is going on. You will be sniffing the traffic of your External services. For this, you can use Wireshark which has an integration with GNS3 or tcpdump from the machines.

1. Access your Web's http page from outside and capture the traffic between the gateway and the bridged interface.

Can you see what is being sent?

What kind of information can you get from this?

What do the headers mean?

2. SSH to the Admin from outside and capture the traffic (make sure to start capturing before connecting to the server).

Can you see what is being sent?

What kind of information can you get from this?

What are the names of the ciphers used?

3. Configure the Burp suite as a proxy on your machine and intercept your HTTP traffic.

Show that you can modify the contents by changing something in the request.

Why are you able to do this here and not in an SSH connection?

Task 3 - IPv6

1. Configure IPv6 from the Web Server to the Worker. This includes IPs on the servers and the default gateway.

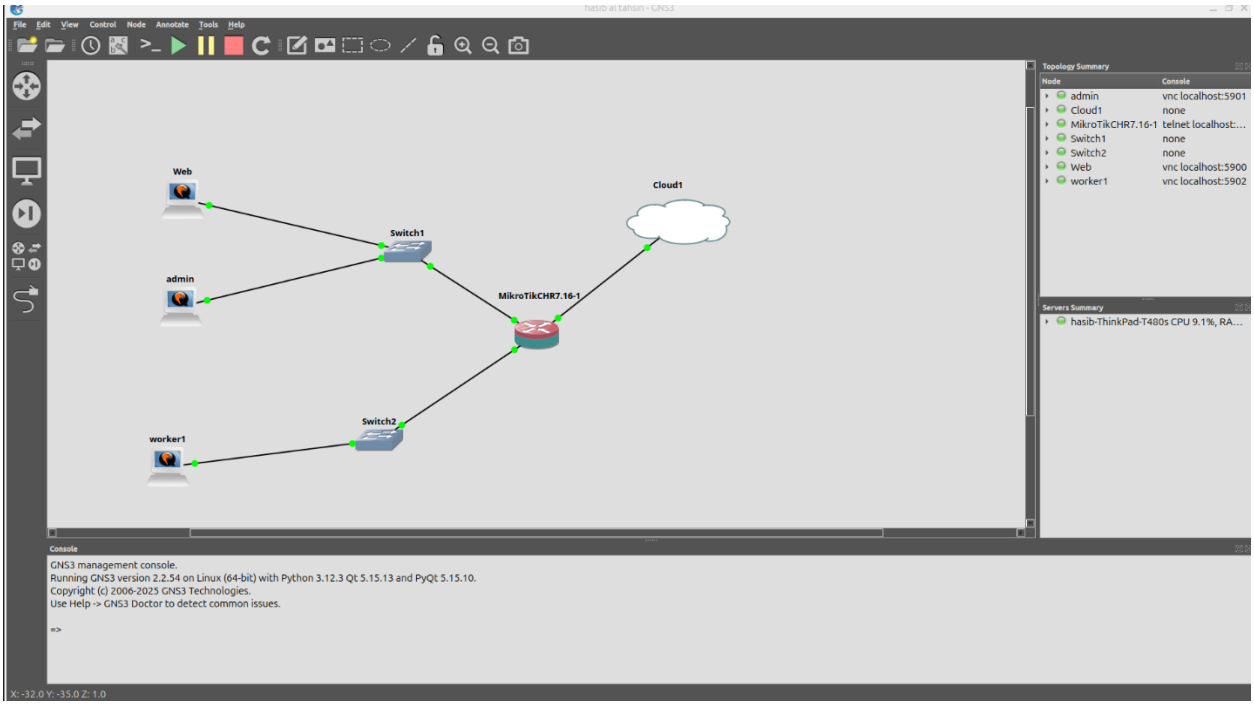
2. Access the Web's http page using IPv6 from Admin while capturing the traffic again. Can you see the difference? What's the difference in packages? Explain.

3. Practice in IPv6 addresses compressing and decompressing. Write your used IPv6 addresses both in full and compact mode. Provide the calculation chain.

Attach your IPv6 captures in a folder captures with your report

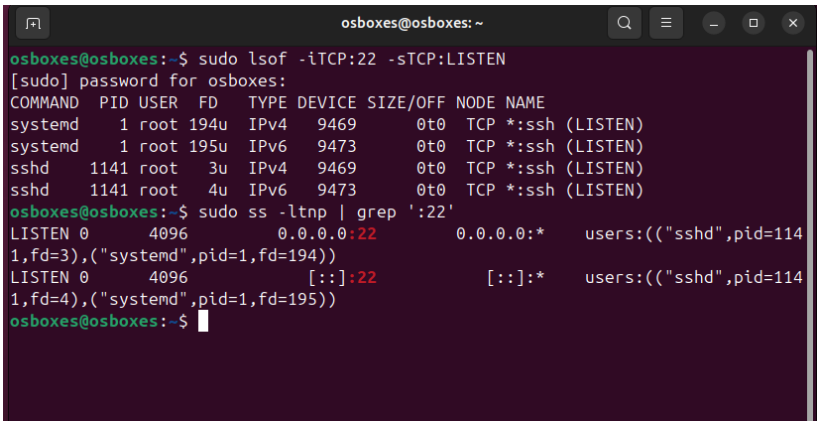
Task 1 Solution:

I installed gns3 at my ubuntu desktop. Then I did setup ubuntu cloud guest where I used **Ubuntu 25.10 desktop setup [1].** The cloud is connected to the router through the Ethernet 4 port using vibr0. The gns3 workflow and components with diagram is below:

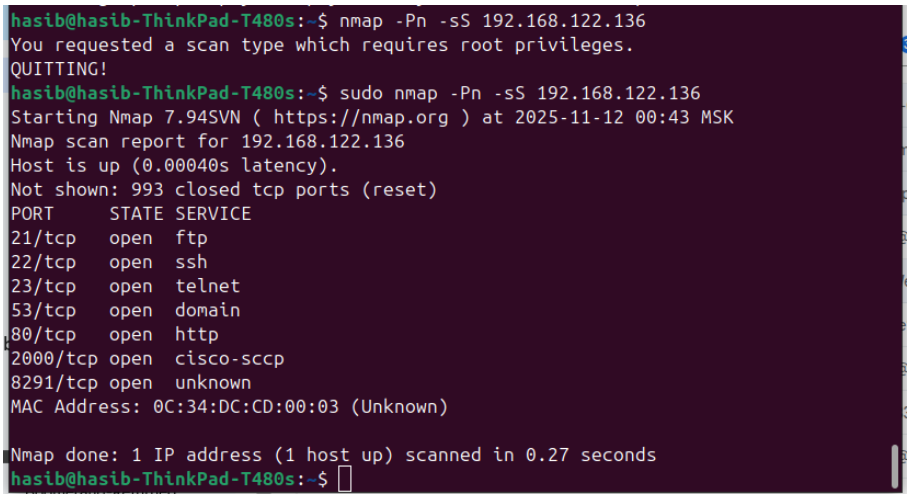


Checking open ports in Admin and Web (ssh=22, http=80)

In Admin:



On Host:



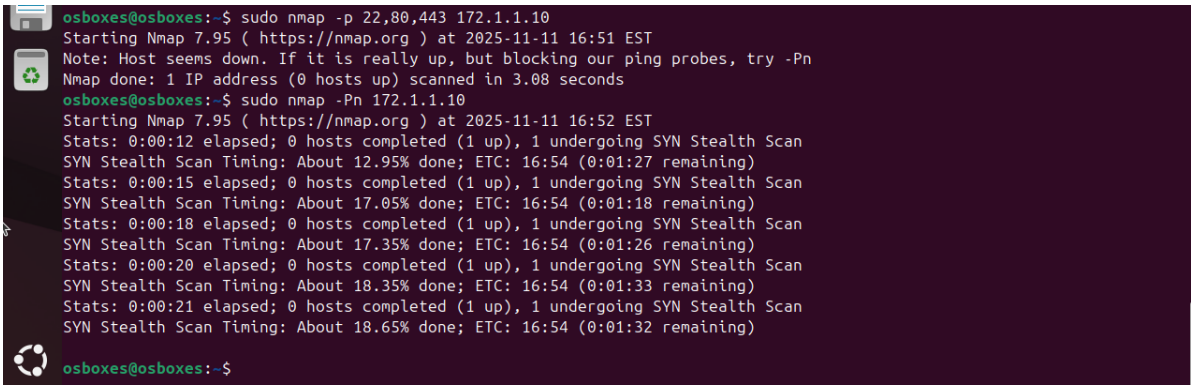
The output shows sshd (22) and apache2 (80) LISTEN. For example: ftp, ssh, telenet, http, cisco-sccp etc. [2].

- Explains that port 22 is the default TCP port for the OpenSSH Server (sshd) service (remote shell).

Admin to Worker (default scan):

Admin terminal: Worker's IPv4 (172.1.1.10)

Showing what is open [3].



ICMP block on Worker + SSH port above 10000 [2], [3]

On Worker:

ICMP block (direct iptables)

- Change SSH port (example 22022)
- Allow new port with firewall (ufw used)

```
osboxes@osboxes:~$ sudo sed -i 's/^#\?Port .*/Port 22022/' /etc/ssh/sshd_config
[sudo] password for osboxes:
osboxes@osboxes:~$ sudo systemctl restart ssh
osboxes@osboxes:~$ sudo ufw allow 22022/tcp
Rules updated
Rules updated (v6)
osboxes@osboxes:~$ sudo ufw deny 22/tcp
Rules updated
Rules updated (v6)
osboxes@osboxes:~$
```

“Default” nmap from Admin (with ping-discovery)

- Force all port scan (except ping, all ports)
- Specific: nmap -Pn -sS -p 22022 172.1.1.10. I have seen the new SSH port (22022) here. (nmap host discovery is not just ping.)

Gather information about Web servers (ssh+http) From Admin:

- 10.1.1.20 = assumed to be the local IP of the Web. This gives the raw scan output [4].

```
osboxes@osboxes:~$ nmap -Pn -sS -p -T4 172.1.1.10
You requested a scan type which requires root privileges.
QUITTING!
osboxes@osboxes:~$ sudo nmap -Pn -sS -T4 172.1.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-11 17:26 EST
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 14.00% done; ETC: 17:26 (0:00:37 remaining)
Stats: 0:00:07 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 17.95% done; ETC: 17:26 (0:00:32 remaining)
Stats: 0:00:08 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 21.90% done; ETC: 17:26 (0:00:29 remaining)

osboxes@osboxes:~$ sudo nmap -sC -sV -p 22,80 10.1.1.20
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-11 17:27 EST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.76 seconds
osboxes@osboxes:~$ sudo nmap -Pn -sC -sV -p 22,80 10.1.1.20
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-11 17:27 EST
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 1 undergoing ARP Ping Scan
ARP Ping Scan Timing: About 99.99% done; ETC: 17:27 (0:00:00 remaining)
Nmap done: 1 IP address (0 hosts up) scanned in 1.65 seconds
```

Ping through the Router:

I did ping the router at different websites like: googe.com, youtube.com (where packet-loss was 0%)

```
[admin@MikroTik] > /ping 8.8.8.8
  SEQ HOST                                     SIZE TTL TIME       STATUS
    0 8.8.8.8                                    56 111 45ms778us
    1 8.8.8.8                                    56 111 46ms344us
    2 8.8.8.8                                    56 111 45ms966us
    3 8.8.8.8                                    56 111 46ms724us
    4 8.8.8.8                                    56 111 46ms402us
    5 8.8.8.8                                    56 111 45ms313us
    6 8.8.8.8                                    56 111 46ms588us
    7 8.8.8.8                                    56 111 46ms213us
    8 8.8.8.8                                    56 111 47ms65us
    sent=9 received=9 packet-loss=0% min-rtt=45ms313us avg-rtt=46ms265us
   max-rtt=47ms65us

[admin@MikroTik] > /ping google.com
  SEQ HOST                                     SIZE TTL TIME       STATUS
    0 216.239.38.120                             56 110 40ms876us
    1 216.239.38.120                             56 110 41ms118us
    2 216.239.38.120                             56 110 40ms724us
    3 216.239.38.120                             56 110 42ms742us
    4 216.239.38.120                             56 110 41ms349us
    5 216.239.38.120                             56 110 41ms28us
    6 216.239.38.120                             56 110 41ms224us
    7 216.239.38.120                             56 110 40ms520us
    8 216.239.38.120                             56 110 41ms168us
    sent=9 received=9 packet-loss=0% min-rtt=40ms520us avg-rtt=41ms194us
   max-rtt=42ms742us

[admin@MikroTik] > /ping youtube.com
  SEQ HOST                                     SIZE TTL TIME       STATUS
    0 64.233.164.91                              56 108 43ms611us
    1 64.233.164.91                              56 108 43ms188us
    2 64.233.164.91                              56 108 43ms962us
    3 64.233.164.91                              56 108 40ms541us
    4 64.233.164.91                              56 108 40ms765us
    sent=5 received=5 packet-loss=0% min-rtt=40ms541us avg-rtt=42ms413us
   max-rtt=43ms962us

[admin@MikroTik] >
```

Task 2 Solution:

Traffic Captures

Shows what HTTP/SSH traffic looks like so capture with GNS3/wireshark or tcpdump.

- Web HTTP access + capture “from outside”
  port-forward (dst-nat) on gateway
  Web local IP 10.1.1.20

In this step, port-forwarding (dst-nat) has been configured as a gateway on the MikroTik router, the purpose of which is to send HTTP traffic coming from the outside to a Web server inside the local network. The local IP of the Web server is 10.1.1.20. Port forwarding is basically a process where the router itself receives traffic but redirects it to a specific server inside. The cloud is connected to the router through the Ethernet 4 port using vibr0. According to this configuration, whenever someone sends an HTTP (i.e. TCP port 80) request from the outside to the router's ether4 interface, the router changes the destination IP of that packet and sends it to the local Web server address 10.1.1.20. As a result, external users can view the content of the internal Web server using the router's public IP, but do not need to know the private IP of the actual server. Here the NAT rule is only effective for HTTP traffic and it explicitly specifies that traffic coming in on the WAN-side interface will only be forwarded to the internal Web server. This way the router acts as a secure gateway and makes internal services accessible from the outside, but keeps its own private IP hidden.

```
[admin@MikroTik] > # Web  IP  10.1.1.20
[admin@MikroTik] > /ip firewall nat add chain=dstnat in-interface=ether4 protocol=tcp dst-port=80 \
\...   action=dst-nat to-addresses=10.1.1.20 to-ports=80 comment="HTTP to Web"
[admin@MikroTik] >
[admin@MikroTik] >
```

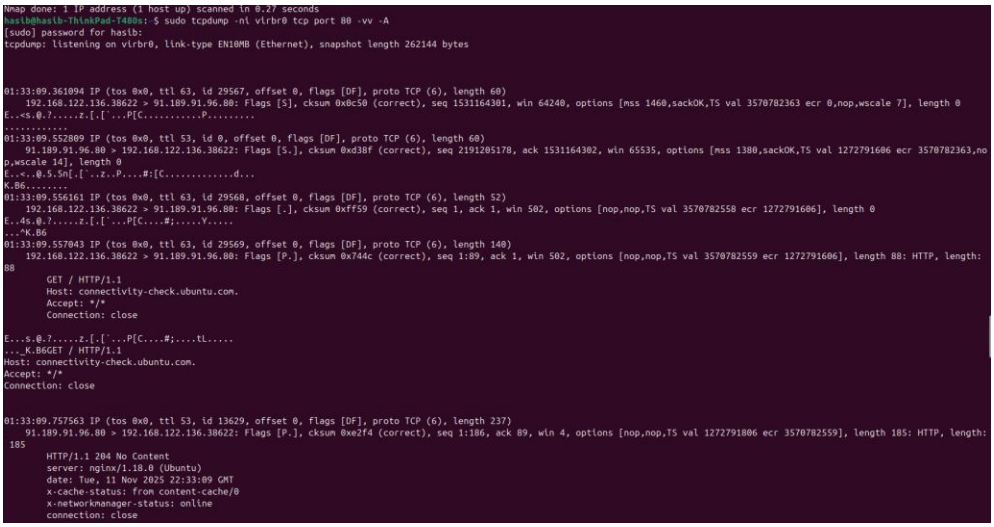
- **Start capturing**

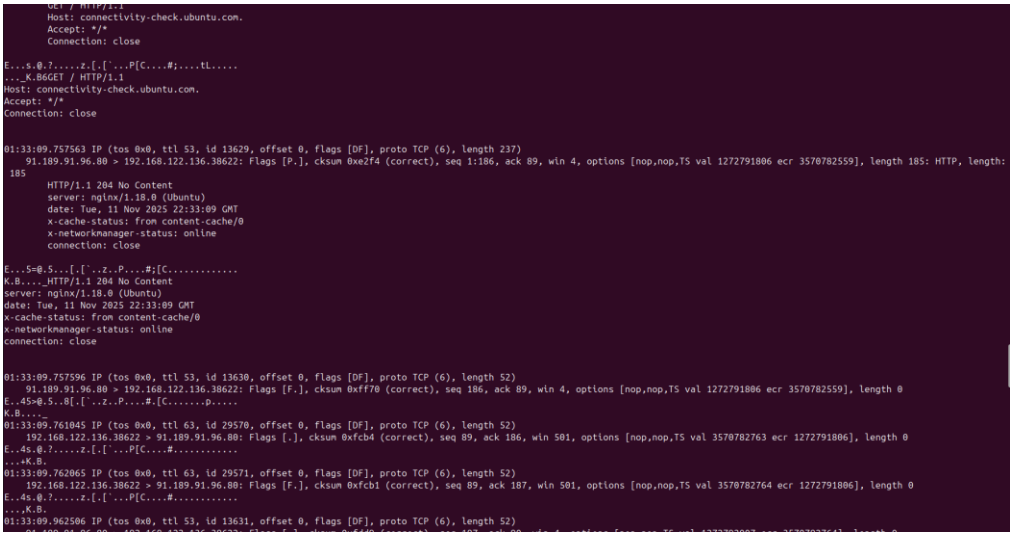
I did right-click on Router ether4 to Cloud1 in the GNS3 canvas to Start capture.

In this step, capture has been started to monitor network traffic, so that HTTP traffic (TCP port 80) passing through the router can be analyzed. The first method uses GNS3, where packets are captured directly on the link between the router and the Cloud. Right-clicking on the connection line between ether4 and Cloud1 of the Router in the GNS3 canvas and selecting Start Capture automatically opens Wireshark and all packets passing through that link can be seen immediately. Through this, it is easy to observe which HTTP requests the router is accepting, how the packet looks before or after NAT, and how the IPv4/IPv6 headers are. tcpdump has been used directly on the host machine, which captures HTTP traffic flowing on the virbr0 interface. tcpdump is a very powerful CLI-based packet capture tool, which shows TCP packets on port 80 in detail (-vv) and in ASCII form (-A), so that the HTTP header and payload can be easily analyzed. The virbr0

interface typically acts as a virtual bridge, so it can see browser traffic, packets after NAT, or requests sent towards the router. Both methods are useful for network debugging, observing NAT behavior, and analyzing HTTP requests/responses [5], [6].

On Host:





```
E..4mp@.?..E..z..}.`.&.PB...z..V....^.....
H&...>.

01:33:10.199740 IP (tos 0x0, ttl 63, id 28017, offset 0, flags [DF], proto TCP (6), length 140)
    192.168.122.136.49190 > 185.125.190.96.80: Flags [P.], cksum 0xd397 (correct), seq 1:89, ack 1, win 502, options [nop,nop,TS val 1210486013 ecr 2319325962], length 88: HTTP, length
: 88
        GET / HTTP/1.1
        Host: connectivity-check.ubuntu.com.
        Accept: */*
        Connection: close

E...mq@.?.....z..}.`.&.PB...z..V...........
H&...>.
GET / HTTP/1.1
Host: connectivity-check.ubuntu.com.
Accept: */*
Connection: close


01:33:10.270707 IP (tos 0x0, ttl 55, id 25429, offset 0, flags [DF], proto TCP (6), length 237)
    185.125.190.96.80 > 192.168.122.136.49190: Flags [P.], cksum 0x4bbc (correct), seq 1:186, ack 89, win 4, options [nop,nop,TS val 2319326037 ecr 1210486013], length 185: HTTP, lengt
h: 185
        HTTP/1.1 204 No Content
        server: nginx/1.18.0 (Ubuntu)
        date: Tue, 11 Nov 2025 22:33:10 GMT
        x-cache-status: from content-cache/0
        x-networkmanager-status: online
        connection: close

E...cU@.7.,..}.`..z..P.&z..VB..!....K......
i>.UH&..HTTP/1.1 204 No Content
server: nginx/1.18.0 (Ubuntu)
date: Tue, 11 Nov 2025 22:33:10 GMT
x-cache-status: from content-cache/0
x-networkmanager-status: online
connection: close


01:33:10.270739 IP (tos 0x0, ttl 55, id 25430, offset 0, flags [DF], proto TCP (6), length 52)
    185.125.190.96.80 > 192.168.122.136.49190: Flags [F.], cksum 0x5f39 (correct), seq 186, ack 89, win 4, options [nop,nop,TS val 2319326037 ecr 1210486013], length 0
E..4cV@.7.`_.}.`..z..P.&z...B..!....._9.....
i>.UH&..
01:33:10.273887 IP (tos 0x0, ttl 63, id 28018, offset 0, flags [DF], proto TCP (6), length 52)
    192.168.122.136.49190 > 185.125.190.96.80: Flags [.], cksum 0x5cff (correct), seq 89, ack 186, win 501, options [nop,nop,TS val 1210486087 ecr 2319326037], length 0
```

```
01:33:10.879426 IP (tos 0x0, ttl 63, id 44302, offset 0, flags [DF], proto TCP (6), length 60)
    192.168.122.136.45246 > 91.189.91.97.80: Flags [S], cksum 0x0f63 (correct), seq 2894937005, win 64240, options [mss 1460,sackOK,TS val 1807173212 ecr 0,nop,wscale 7], length 0
E..<..@.?..^..z.[.[a...P..;..........c.........
k.F\........
01:33:10.884105 IP (tos 0x0, ttl 53, id 0, offset 0, flags [DF], proto TCP (6), length 60)
    91.189.91.97.80 > 192.168.122.136.45246: Flags [S.], cksum 0x01a9 (correct), seq 3165065202, ack 2894937006, win 65535, options [mss 1380,sackOK,TS val 3898233838 ecr 1807173212,no
p,wscale 14], length 0
E..<..@.5.5m[.[a..z..P.......;...........d...
.ZS.k.F\....
01:33:10.887492 IP (tos 0x0, ttl 63, id 44303, offset 0, flags [DF], proto TCP (6), length 52)
    192.168.122.136.45246 > 91.189.91.97.80: Flags [.], cksum 0x2d66 (correct), seq 1, ack 1, win 502, options [nop,nop,TS val 1807173420 ecr 3898233838], length 0
E..4..@.?..@..z.[.[a...P..;..........f.....
k.G,.ZS.
01:33:10.888285 IP (tos 0x0, ttl 63, id 44304, offset 0, flags [DF], proto TCP (6), length 140)
    192.168.122.136.45246 > 91.189.91.97.80: Flags [P.], cksum 0xa258 (correct), seq 1:89, ack 1, win 502, options [nop,nop,TS val 1807173421 ecr 3898233838], length 88: HTTP, length:
88
        GET / HTTP/1.1
        Host: connectivity-check.ubuntu.com.
        Accept: */*
        Connection: close

E.....@.?.....z.[.[a...P..;..........X.....
k.G-.ZS.GET / HTTP/1.1
Host: connectivity-check.ubuntu.com.
Accept: */*
Connection: close


01:33:11.029386 IP (tos 0x0, ttl 53, id 11216, offset 0, flags [DF], proto TCP (6), length 237)
    91.189.91.97.80 > 192.168.122.136.45246: Flags [P.], cksum 0x17f7 (correct), seq 1:186, ack 89, win 4, options [nop,nop,TS val 3898234047 ecr 1807173421], length 185: HTTP, length:
 185
        HTTP/1.1 204 No Content
        server: nginx/1.18.0 (Ubuntu)
        date: Tue, 11 Nov 2025 22:33:11 GMT
        x-cache-status: from content-cache/1
        x-networkmanager-status: online
        connection: close

E...+.@.5.&.[.[a..z..P........<............
.ZT.k.G-HTTP/1.1 204 No Content
server: nginx/1.18.0 (Ubuntu)
date: Tue, 11 Nov 2025 22:33:11 GMT
x-cache-status: from content-cache/1
x-networkmanager-status: online
connection: close
```

```
01:33:11.029386 IP (tos 0x0, ttl 53, id 11216, offset 0, flags [DF], proto TCP (6), length 237)
    91.189.91.97.80 > 192.168.122.136.45246: Flags [P.], cksum 0x17f7 (correct), seq 1:186, ack 89, win 4, options [nop,nop,TS val 3898234047 ecr 1807173421], length 185: HTTP, length:
 185
        HTTP/1.1 204 No Content
        server: nginx/1.18.0 (Ubuntu)
        date: Tue, 11 Nov 2025 22:33:11 GMT
        x-cache-status: from content-cache/1
        x-networkmanager-status: online
        connection: close

E...+.@.5.&.[.[a..z..P........<............
.ZT.k.G-HTTP/1.1 204 No Content
server: nginx/1.18.0 (Ubuntu)
date: Tue, 11 Nov 2025 22:33:11 GMT
x-cache-status: from content-cache/1
x-networkmanager-status: online
connection: close


01:33:11.029422 IP (tos 0x0, ttl 53, id 11217, offset 0, flags [DF], proto TCP (6), length 52)
    91.189.91.97.80 > 192.168.122.136.45246: Flags [F.], cksum 0x2d74 (correct), seq 186, ack 89, win 4, options [nop,nop,TS val 3898234047 ecr 1807173421], length 0
E..4+.@.5.'.[.[a..z..P........<.....-t.....
.ZT.k.G-
01:33:11.033045 IP (tos 0x0, ttl 63, id 44305, offset 0, flags [DF], proto TCP (6), length 52)
    192.168.122.136.45246 > 91.189.91.97.80: Flags [.], cksum 0x2af4 (correct), seq 89, ack 186, win 501, options [nop,nop,TS val 1807173565 ecr 3898234047], length 0
E..4..@.?..c..z.[.[a...P..<.........*......
k.G..ZT.
01:33:11.033873 IP (tos 0x0, ttl 63, id 44306, offset 0, flags [DF], proto TCP (6), length 52)
    192.168.122.136.45246 > 91.189.91.97.80: Flags [F.], cksum 0x2af1 (correct), seq 89, ack 187, win 501, options [nop,nop,TS val 1807173566 ecr 3898234047], length 0
E..4..@.?..b..z.[.[a...P..<.........*......
k.G..ZT.
01:33:11.193820 IP (tos 0x0, ttl 53, id 11218, offset 0, flags [DF], proto TCP (6), length 52)
    91.189.91.97.80 > 192.168.122.136.45246: Flags [.], cksum 0x2c51 (correct), seq 187, ack 90, win 4, options [nop,nop,TS val 3898234192 ecr 1807173566], length 0
E..4+.@.5.'.[.[a..z..P........<.....,Q.....
.ZUPk.G.


^C
27 packets captured
27 packets received by filter
0 packets dropped by kernel
hasib@hasib-ThinkPad-T480s:~$
```

Task 3 Solution: IPv6

Here, running IPv6 in Web to Router to Worker, IPv6 HTTP test and capture from Admin, and compression and decompression are practiced.

IPv6 Addressing

Router (MikroTik)

This command enables the IPv6 protocol on the router. IPv6 disabled by default on MikroTik routers. By setting disable-ipv6=no, the router is able to handle IPv6 traffic. This enables IPv6 support for the entire router, so that IPv6 addresses can be added to specific interfaces in the next step [7].

ether1

This command assigns an IPv6 address to the ether1 interface. address=2001:db8:10::1/64 means that this is an IPv6 address with the network prefix 2001:db8:10::1, with a subnet size of /64. interface=ether1 indicates that this address is associated with the ether1 interface. advertise=no means that the router does not broadcast this address as an advertisement or Router Advertisement (RA). That is, it will not automatically inform clients about this address. This is used for the admin or server side, where static configuration is used. With this setup, the Admin/Web side (ether1) is connected to the IPv6 network, so that administrative access or web management can be performed [8], [9].

ether3 (Form Worker Side)

Here address=2001:db8:172::1/64 indicates that it is included in the 2001:db8:172::/64 network. interface=ether3 indicates that the address is on the ether3 port. advertise=no means that automatic address advertisement is disabled here as well, the router will not send IPv6 settings to worker or client side devices automatically, but must be set manually. This interface is usually used for worker or internal networks, where IPv6 communication required. But the router will not advertise [9].

```
[admin@MikroTik] > # Web  IP  10.1.1.20
[admin@MikroTik] > /ip firewall nat add chain=dstnat in-interface=ether4 protocol=tcp dst-port=80
\...   action=dst-nat to-addresses=10.1.1.20 to-ports=80 comment="HTTP to Web"
[admin@MikroTik] >
[admin@MikroTik] > /ipv6 settings set disable-ipv6=no
[admin@MikroTik] >
[admin@MikroTik] > # ether1 (Admin/Web )
[admin@MikroTik] > /ipv6 address add address=2001:db8:10::1/64 interface=ether1 advertise=no
[admin@MikroTik] >
[admin@MikroTik] > # ether3 (Worker )
[admin@MikroTik] > /ipv6 address add address=2001:db8:172::1/64 interface=ether3 advertise=no
[admin@MikroTik] >
[admin@MikroTik] >
```

- **Running mitmproxy (to generate CA certificate)**

mitmproxy

Running this opens an interactive window in the terminal. It creates a ~/.mitmproxy folder and creates certificate files there.

- **Running proxy with virbr0 interface**

virbr0 is used (IP: 192.168.122.1), listening as mitmproxy proxy [10], [11].

- **Copying the CA certificate file**

Since the .mitmproxy directory has been created, it is copied to /tmp so that it can be sent to the Web/Worker node later.

Fixing IPv6 forwarding

Results-

1. Running mitmproxy creates ~/.mitmproxy
2. Then /tmp/mitmproxy-ca-cert.pem is copied
3. Proxy 192.168.122.1:8080 is given in Web node/browser
4. Then HTTP/HTTPS test is given with browser or curl
5. Traffic capture can be seen in mitmproxy on host's terminal [12]

```
hasib@hasib-ThinkPad-T480s:~$ sudo echo 'net.ipv6.conf.all.forwarding=1' | sudo
tree -a /etc/sysctl.conf
sudo: tree: command not found
hasib@hasib-ThinkPad-T480s:~$ mitmproxy
hasib@hasib-ThinkPad-T480s:~$ ls ~/.mitmproxy
mitmproxy-ca-cert.cer  mitmproxy-ca-cert.pem  mitmproxy-ca.pem
mitmproxy-ca-cert.p12  mitmproxy-ca.p12       mitmproxy-dhparam.pem
hasib@hasib-ThinkPad-T480s:~$ sudo mitmproxy --listen-host 192.168.122.1 --liste
n-port 8080
hasib@hasib-ThinkPad-T480s:~$
```

```
hasib@hasib-ThinkPad-T480s:~$ sudo nohup mitmproxy --listen-host 192.168.122.1 -
-listen-port 8080 > /tmp/mitmproxy.log 2>&1 &
[2] 22094
hasib@hasib-ThinkPad-T480s:~$ cp ~/.mitmproxy/mitmproxy-ca-cert.pem /tmp/mitmpro
xy-ca-cert.pem
[2]-  Exit 1                  sudo nohup mitmproxy --listen-host 192.168.122.1 -
-listen-port 8080 > /tmp/mitmproxy.log 2>&1
hasib@hasib-ThinkPad-T480s:~$ sudo sysctl -w net.ipv6.conf.all.forwarding=1
echo 'net.ipv6.conf.all.forwarding=1' | sudo tee -a /etc/sysctl.conf
net.ipv6.conf.all.forwarding = 1
net.ipv6.conf.all.forwarding=1
hasib@hasib-ThinkPad-T480s:~$
```

- **Web (interface name ens3; ip -6 a is seen)**
  Router (MikroTik)

The first command used to configure IPv6 on a Linux system is the ip -6 addr add command. This command adds an IPv6 address to the network interface named ens3. Superuser permissions are required to change the network configuration. The ip -6 addr add command indicates that this is a command to add an IPv6 address. 2001:db8:10::10/64 is the specific IPv6 address and subnet prefix, where /64 indicates that the first 64 bits are used as the network ID and the remaining 64 bits are used as the host ID. The dev ens3 command indicates that this IPv6 address has been applied to the network interface named ens3. This adds the IPv6 address 2001:db8:10::10 to the ens3 interface, allowing the system to actively communicate within that IPv6 subnet [13], [14].

ether1 (Admin/Web side)

This command sets a default IPv6 route (Default Gateway) in the system. Here ip -6 route add means to add a new IPv6 route. Default means that this is the default route, all IPv6 traffic that is not on the system's local subnet is sent through this route. via 2001:db8:10::1 indicates that the next destination or gateway of this route is 2001:db8:10::1, which is usually the IPv6 address of the router or gateway device (in this case, the address of the ether1 interface of the MikroTik router). By adding this route, the system can now send traffic through router 2001:db8:10::1 to communicate with other IPv6 networks [13], [15].

ether3 (Worker side)

This setup enables IPv6 connectivity on the ether3 interface and creates a dedicated IPv6 network (2001:db8:172::/64) for devices on the worker side, with automatic address advertisement disabled for security and controlled configuration [7].

```
osboxes@osboxes:~$ sudo ip -6 route add default via 2001:db8:10::1
RTNETLINK answers: No route to host
osboxes@osboxes:~$ sudo ip -6 addr add 2001:db8:10::10/64 dev ens3
[sudo] password for osboxes:
osboxes@osboxes:~$ sudo ip -6 route add default via 2001:db8:10::1
osboxes@osboxes:~$
```

- **IPv6 addressing provided**
  Router (MikroTik)

First, the IPv6 feature of the router is enabled by running the command. By default, IPv6 service is often disabled on MikroTik routers. This command enables the IPv6 protocol, so that the router can process and route IPv6 traffic [7].

ether1 (Admin/Web side)

Here 2001:db8:10::1/64 is the IPv6 address and subnet prefix. It is added to the ether1 interface, which is used as the administrative (Admin) or web access side. advertise=no indicates that the router will not send Router Advertisement (RA) on this interface, i.e. clients will not be automatically assigned IPv6 addresses; communication is only possible with manual configuration [7], [16].

ether3 (Worker side)

This adds the IPv6 address 2001:db8:172::1/64 to the ether3 interface. This is used for the worker network portion. Here too advertise=no is set, so that automatic IPv6 address distribution is disabled and the network administrator can maintain full control [7], [16].

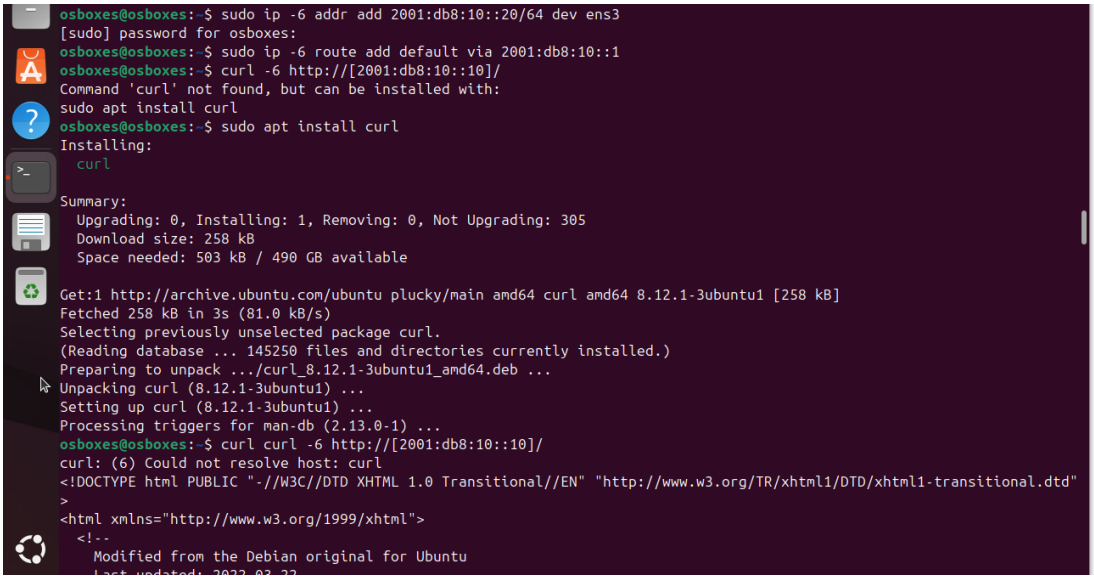
```
[admin@MikroTik] > # Web  IP  10.1.1.20
[admin@MikroTik] > /ip firewall nat add chain=dstnat in-interface=ether4 protocol=tcp dst-port=80 \
\...   action=dst-nat to-addresses=10.1.1.20 to-ports=80 comment="HTTP to Web"
[admin@MikroTik] >
[admin@MikroTik] > /ipv6 settings set disable-ipv6=no
[admin@MikroTik] >
[admin@MikroTik] > # ether1 (Admin/Web )
[admin@MikroTik] > /ipv6 address add address=2001:db8:10::1/64 interface=ether1 advertise=no
[admin@MikroTik] >
[admin@MikroTik] > # ether3 (Worker )
[admin@MikroTik] > /ipv6 address add address=2001:db8:172::1/64 interface=ether3 advertise=no
[admin@MikroTik] >
[admin@MikroTik] >
```

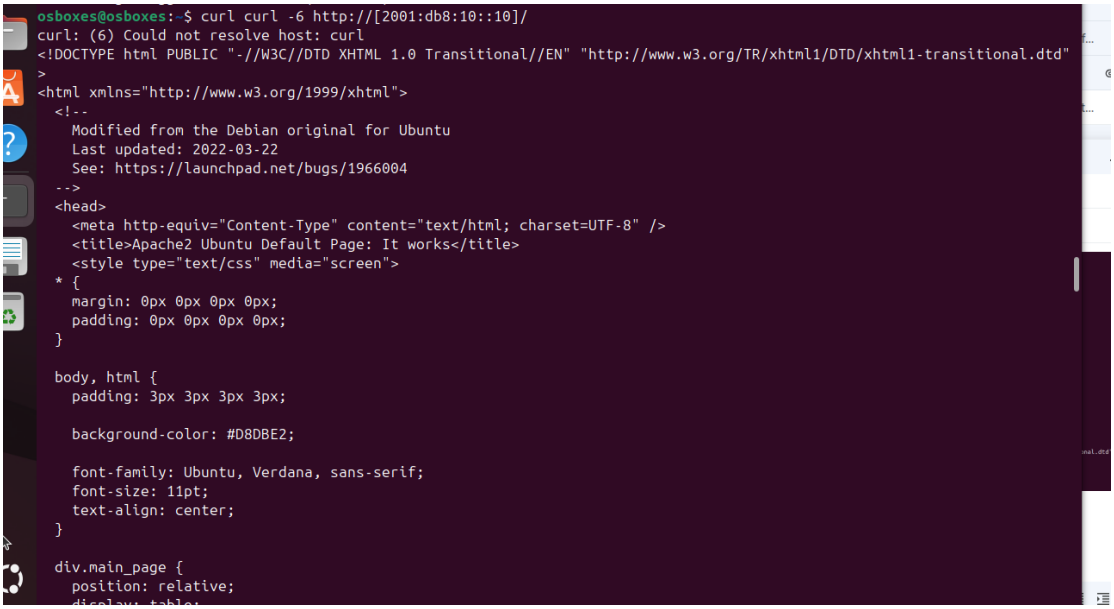
Admin (for Web access via IPv6)

In this section, the Web access using IPv6 is setup and tested from the Admin side. First, the IPv6 address and default route are set on the Admin machine. This adds the IPv6 address 2001:db8:10::20/64 to the interface named ens3. The /64 part indicates the subnet prefix, which is the standard size for IPv6 networks.

Next, the default route is set, which tells the system that the router with the address 2001:db8:10::1 (which is the ether1 interface of MikroTik) is used as the gateway when sending any external IPv6 traffic. This is the default path for IPv6 traffic [13].

IPv6 connectivity was then tested by sending an HTTP request to the web server running at address 2001:db8:10::10 using the IPv6 protocol (-6). The IPv6 address is enclosed in square brackets ([ ]) as required by the URL standard for IPv6 syntax. A successful response from this command indicates that IPv6-based web access from the Admin machine is working properly. As a result, traffic was captured on the router using the torch tool or the GNS3 capture feature. This observation shows the IPv6 packet headers. IPv6 traffic is flowing properly and the packet headers are in IPv6 format [17], [18].





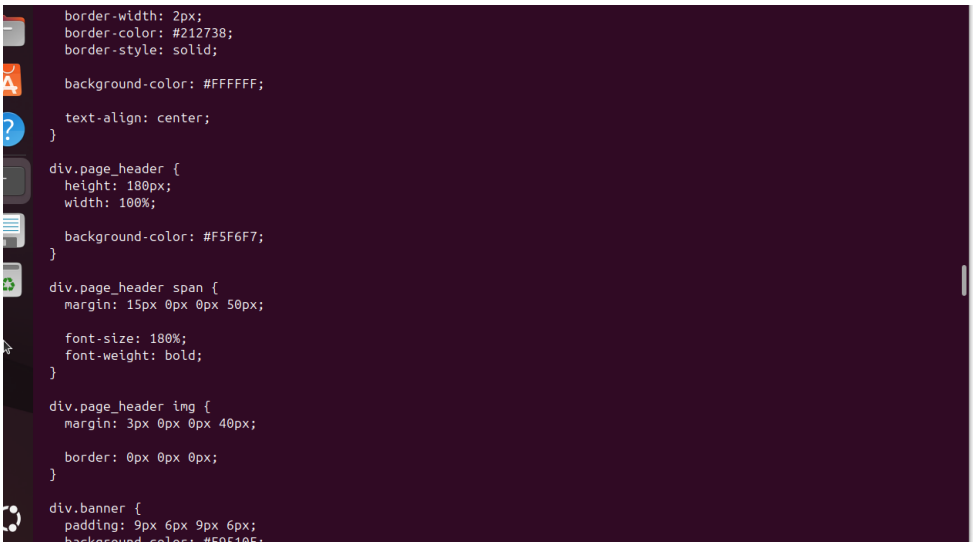
```
osboxes@osboxes:~$ curl curl -6 http://[2001:db8:10::10]/
curl: (6) Could not resolve host: curl
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"
>
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;

    background-color: #D8DBE2;

    font-family: Ubuntu, Verdana, sans-serif;
    font-size: 11pt;
    text-align: center;
  }

  div.main_page {
    position: relative;
    display: table;
```



```
    border-width: 2px;
    border-color: #212738;
    border-style: solid;

    background-color: #FFFFFF;

    text-align: center;
  }

  div.page_header {
    height: 180px;
    width: 100%;

    background-color: #F5F6F7;
  }

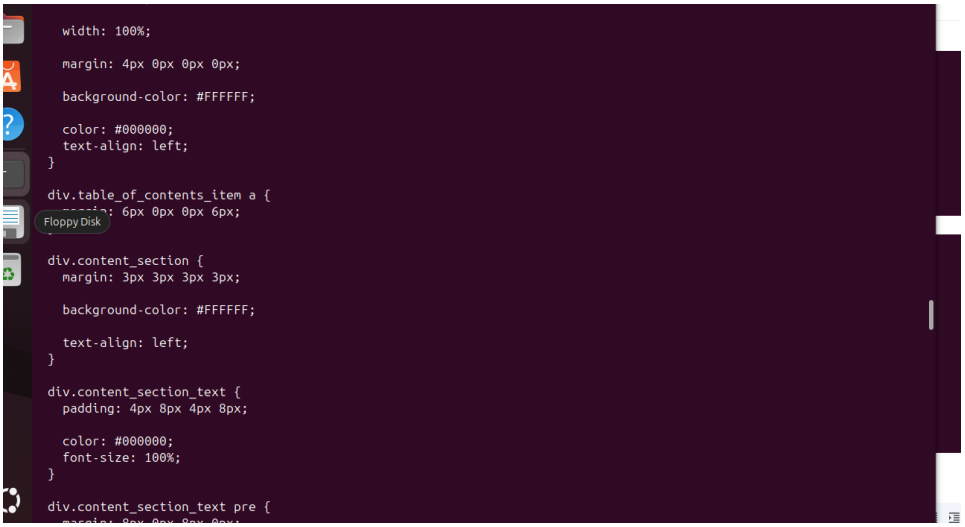
  div.page_header span {
    margin: 15px 0px 0px 50px;

    font-size: 180%;
    font-weight: bold;
  }

  div.page_header img {
    margin: 3px 0px 0px 40px;

    border: 0px 0px 0px;
  }

  div.banner {
    padding: 9px 6px 9px 6px;
    background-color: #E8E19F;
```



```
    width: 100%;

    margin: 4px 0px 0px 0px;

    background-color: #FFFFFF;

    color: #000000;
    text-align: left;
  }

  div.table_of_contents_item a {
    margin: 6px 0px 0px 6px;
  }

  div.content_section {
    margin: 3px 3px 3px 3px;

    background-color: #FFFFFF;

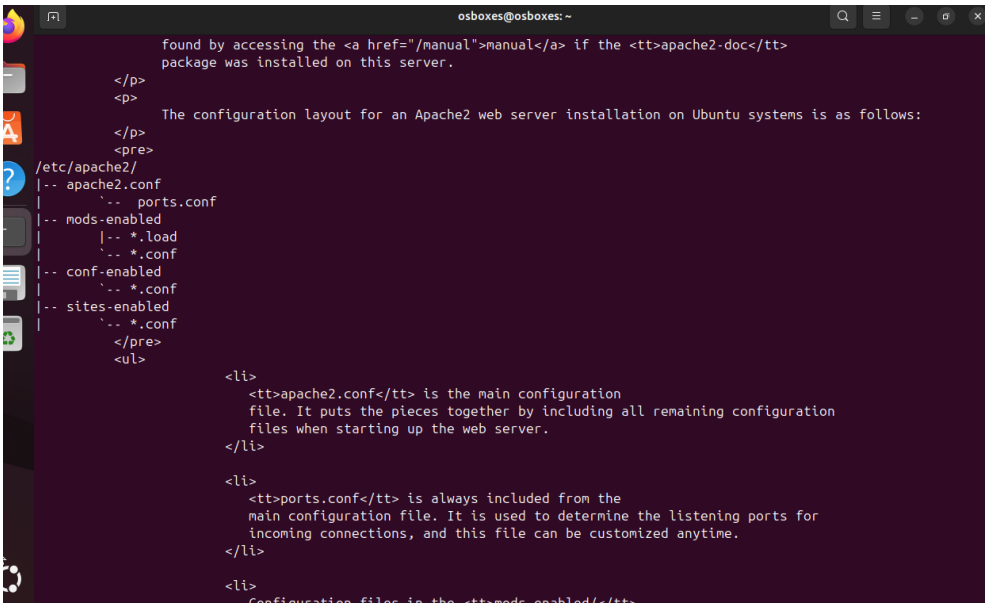
    text-align: left;
  }

  div.content_section_text {
    padding: 4px 8px 4px 8px;

    color: #000000;
    font-size: 100%;
  }

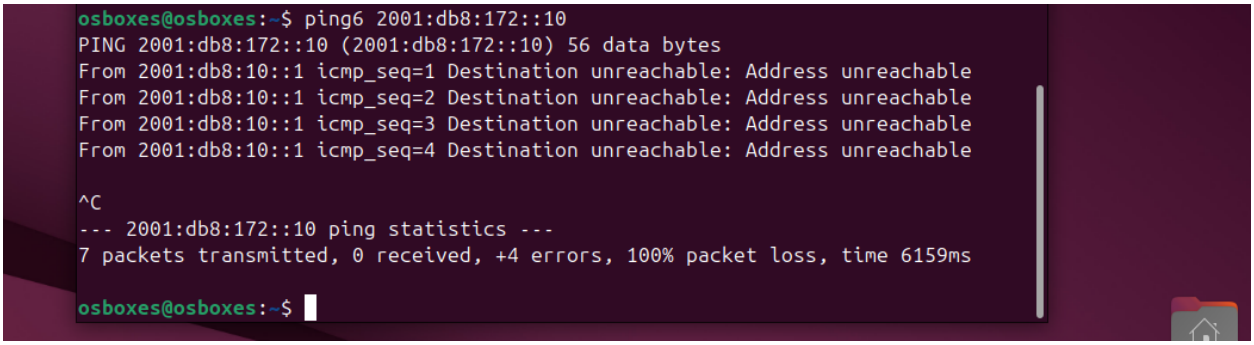
  div.content_section_text pre {
    margin: 8px 0px 8px 0px;
```

Floppy Disk





Web PC has been pinged and ping6 has been performed, showing that Worker is reachable.



- **MikroTik - torch (instant flow summary)**

torch quickly shows which source/destination IPv6 address the traffic is going to and which L4 protocol. But it does not dissect the full four IPv6-fields - it shows the flow level.

In RouterOS shell:

Per line shows src-address dst-address proto src-port dst-port bps etc. IPv6 packets are shown in IPv6 format as src-address and dst-address.

MikroTik - had to make pcap from sniffer where the host can be used to see the full IPv6 header with Wireshark/tshark.

Torch is not enough in RouterOS, sniffer (PCAP) is used to see the full header [19], [20].

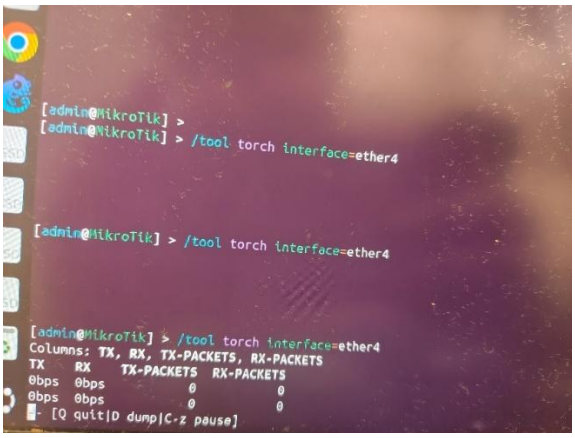
Creating pcap in RouterOS

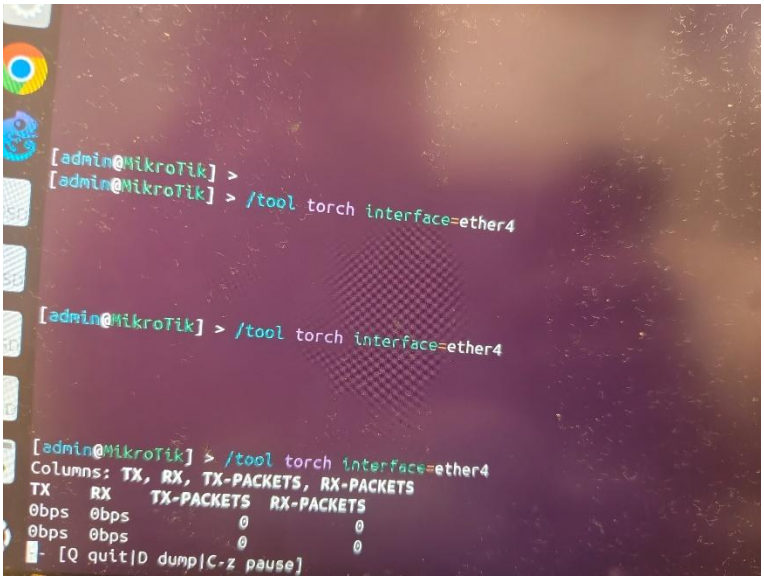
In this step, a pcap file has been created to capture IPv6 traffic in MikroTik RouterOS. The router's sniffer tool (packet capture) has been configured through the command.

file-name=ether4_ipv6.pcap indicates that the captured data will be saved to a file, which will be named ether4_ipv6.pcap. This file is stored in the router's file system. interface=ether4 means that the capture applies only to traffic on the ether4 interface. streaming-enabled=no means that the captured data is not streamed live; rather, it is stored in a file inside the router. With filter-ipv6=yes, the IPv6 packet filter has been enabled, meaning that only IPv6 traffic is recorded in the pcap file. With these settings, the router is ready to capture only IPv6 traffic. Then, the command /tool sniffer start is run to start the capture. After running this command, the router starts monitoring IPv6 traffic on the specified interface (ether4) and writes that traffic to the pcap file [20].

Now, for some time, IPv6 traffic is generated on the network, by sending ping6 from the host or worker machine, or by browsing the IPv6-based web. The router captures and stores this traffic. When the packets are collected, the capture is stopped. This stops the router from recording any new packets, and the pcap file is completely saved. Then, the file list is displayed, which shows a list of all the files present in the router's file system. There, a file named ether4_ipv6.pcap is listed, which proves that the IPv6 traffic was successfully captured [19], [20].





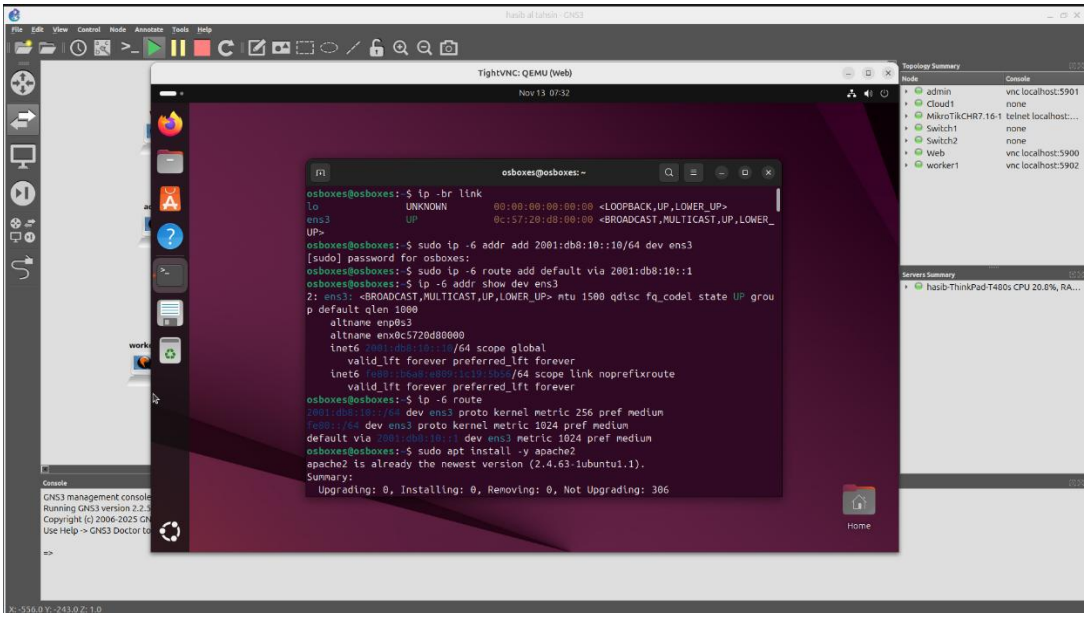


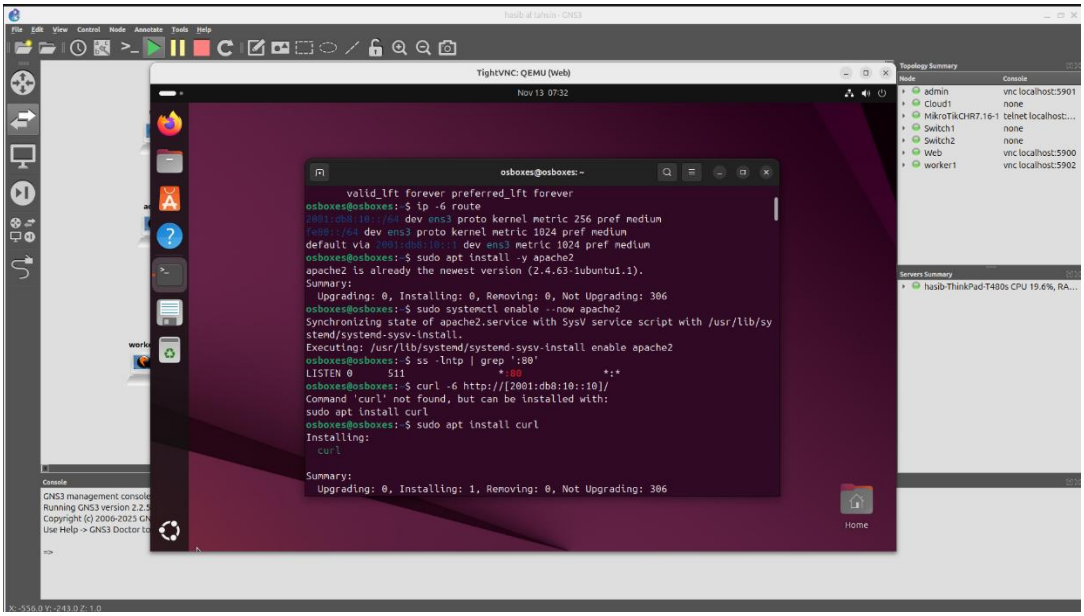
The capture was not successful here, so I tried again, using wireshark.

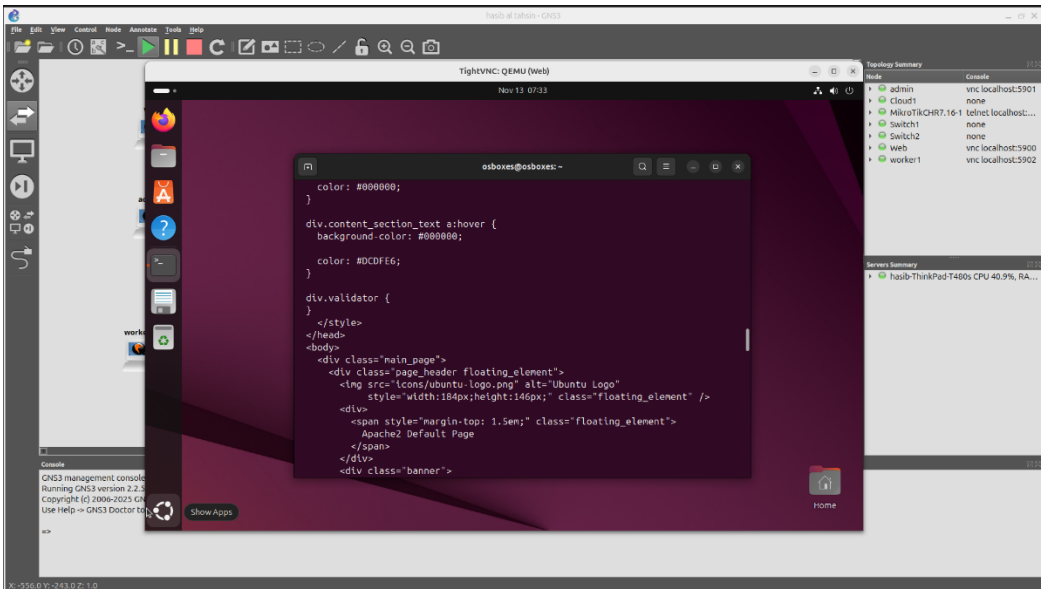
Since IPv6 is enabled in my RouterOS, I did not have to do a new setup.

When setting up a web server, it is very important to add the IPv6 configuration and default gateway, so that the server can communicate with the network properly. First, open the terminal in the Web VM and check the interface name with the ip -br link command, which is usually ens3. Then, the sudo ip -6 addr add 2001:db8:10::10/64 dev ens3 command is used to give the server an IPv6 address. This adds the specified IPv6 address to the server's network interface. Then, the sudo ip -6 route add default via 2001:db8:10::1 command is used to set the default gateway, so that the server can send traffic to another network. After doing all this, the ip -6 addr show dev ens3 and ip -6 route are used to verify whether the configuration has been added correctly. The Apache web server was not installed, so it was installed with sudo apt update and sudo apt install -y apache2 and the service was started and autostarted with sudo systemctl enable --now apache2 . The server

was listened to on port 80 in IPv6 using ss -lntp | grep ':80' to ensure that the HTTP service was running properly [19], [20].



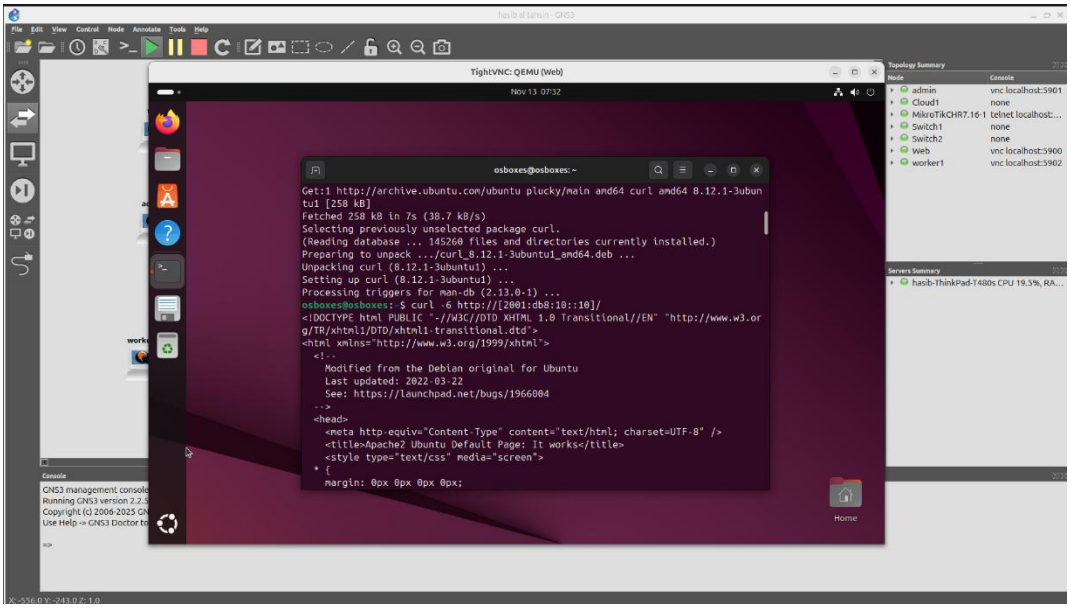


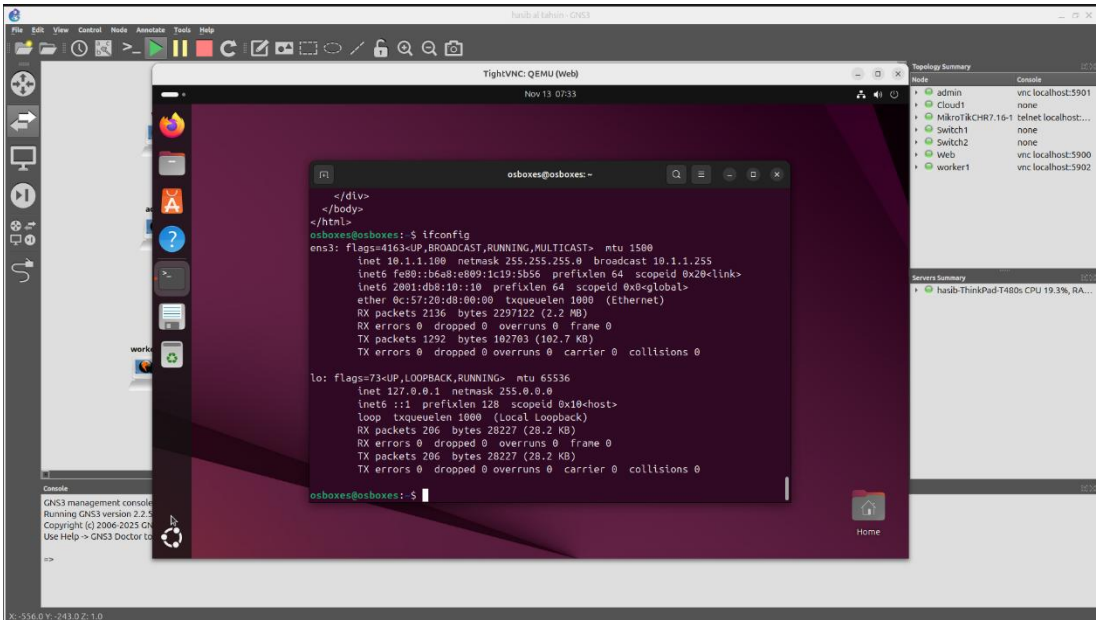

TightVNC: QEMU (Web)
Nov 13 07:33
osboxes@osboxes: ~
color: #000000;
}
div.content_section_text a:hover {
background-color: #000000;
color: #DCDFE6;
}
div.validator {
}
</style>
</head>
<body>

<img src="icons/ubuntu-logo.png" alt="Ubuntu Logo"
style="width:184px;height:146px;" class="floating_element" />

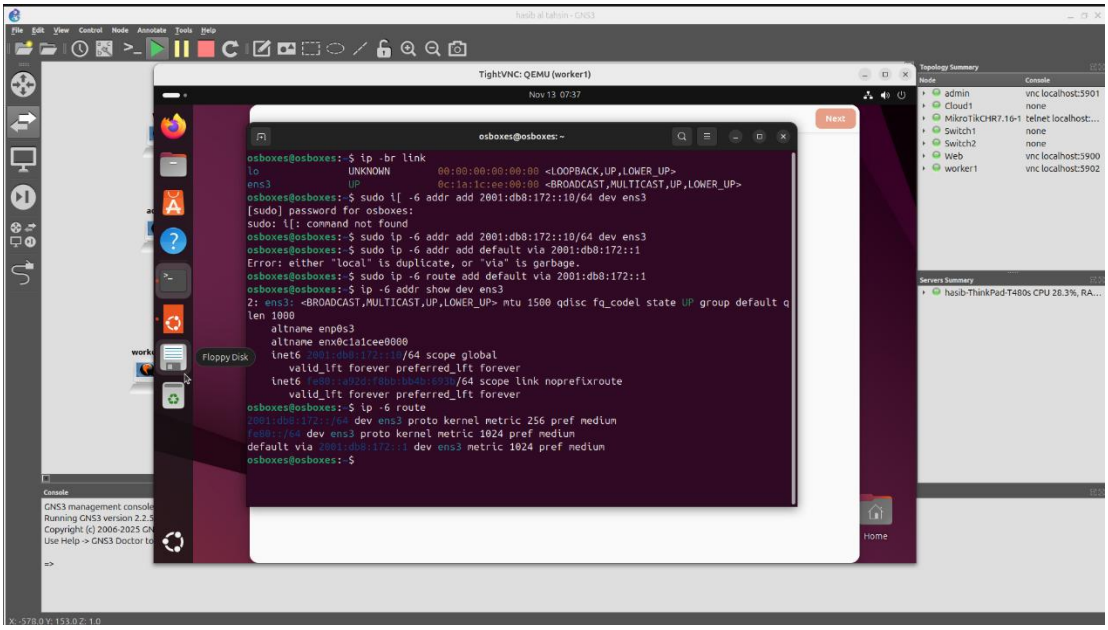
Apache2 Default Page

Show Apps
Home
Topology Summary
Node
Console
admin
vnc localhost:5901
Cloud1
none
MikroTikCHR7.16-1
telnet localhost:...
Switch1
none
Switch2
none
Web
vnc localhost:5900
worker1
vnc localhost:5902
Servers Summary
hasib-ThinkPad-T480s CPU 40.9%, RA...

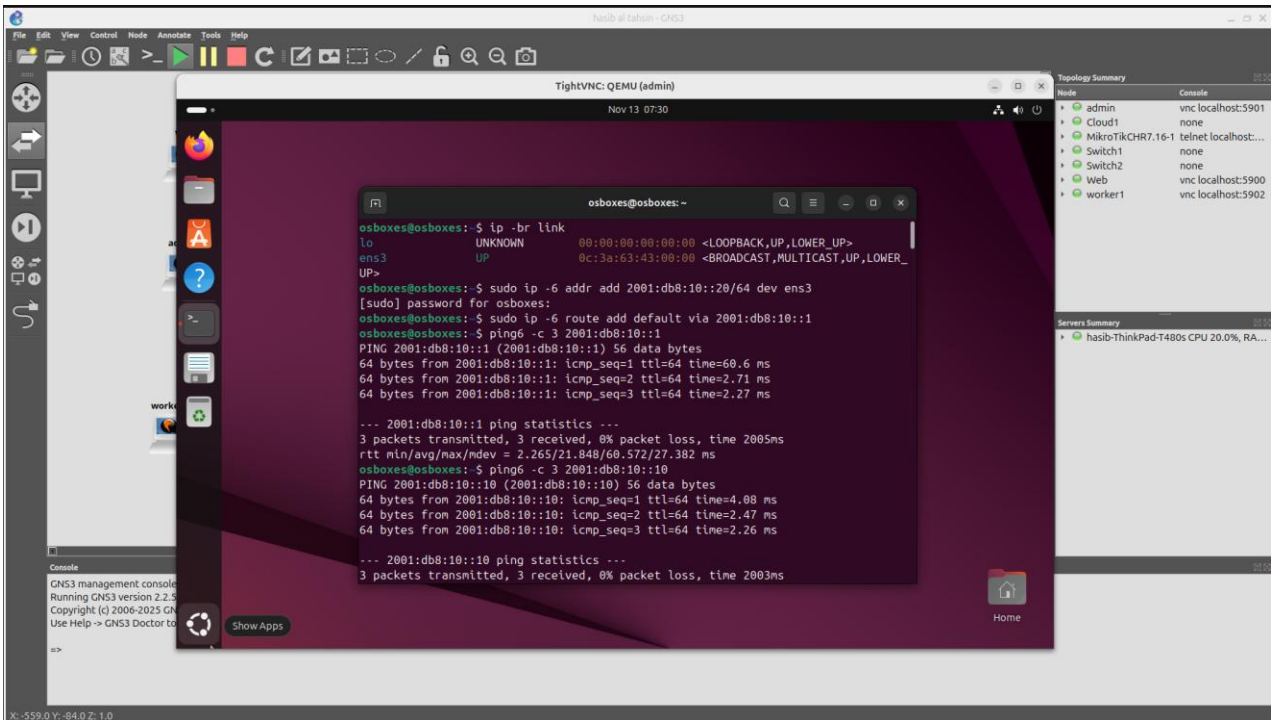

TightVNC: QEMU (Web)
Nov 13 07:32
osboxes@osboxes: ~
Get:1 http://archive.ubuntu.com/ubuntu plucky/main amd64 curl amd64 8.12.1-3ubuntu1 [258 kB]
Fetched 258 kB in 7s (38.7 kB/s)
Selecting previously unselected package curl.
(Reading database ... 145260 files and directories currently installed.)
Preparing to unpack .../curl_8.12.1-3ubuntu1_amd64.deb ...
Unpacking curl (8.12.1-3ubuntu1) ...
Setting up curl (8.12.1-3ubuntu1) ...
Processing triggers for man-db (2.13.0-1) ...
osboxes@osboxes:~$ curl -6 http://[2001:db8:10::10]/
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<!--
Modified from the Debian original for Ubuntu
Last updated: 2022-03-22
See: https://launchpad.net/bugs/1966004
-->
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Apache2 Ubuntu Default Page: It works</title>
<style type="text/css" media="screen">
* {
margin: 0px 0px 0px 0px;
Home
Topology Summary
Node
Console
admin
vnc localhost:5901
Cloud1
none
MikroTikCHR7.16-1
telnet localhost:...
Switch1
none
Switch2
none
Web
vnc localhost:5900
worker1
vnc localhost:5902
Servers Summary
hasib-ThinkPad-T480s CPU 19.5%, RA...

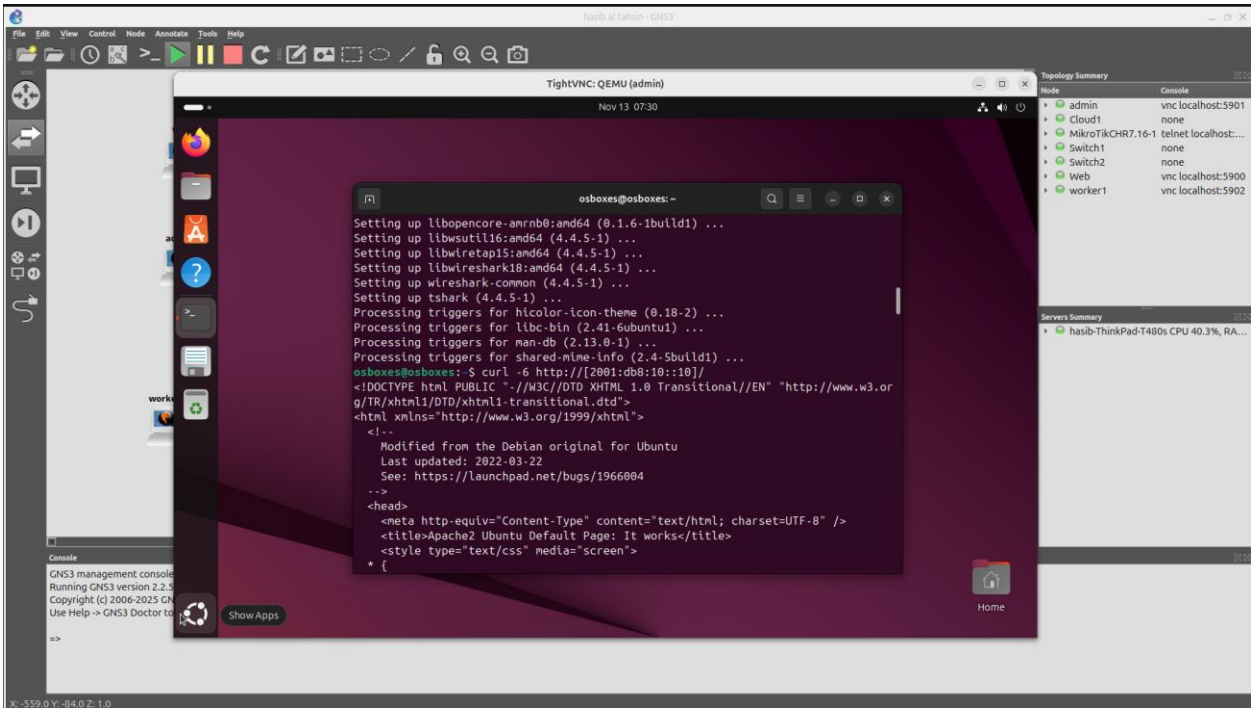


The network interface name was checked using the ip -br link command on the Worker VM, which was ens3. Then, to connect the server to the IPv6 network, the specific IPv6 address was added to the interface using the sudo ip -6 addr add 2001:db8:172::10/64 dev ens3 command. Then, to configure the default gateway, sudo ip -6 route add default via 2001:db8:172::1 was used, which sets the router's ether3 interface as the primary route for sending traffic to the Internet or other networks. The ip -6 addr show dev ens3 command confirmed that the IPv6 address was correctly added, and the ip -6 route command confirmed that the gateway was set correctly [19], [20].



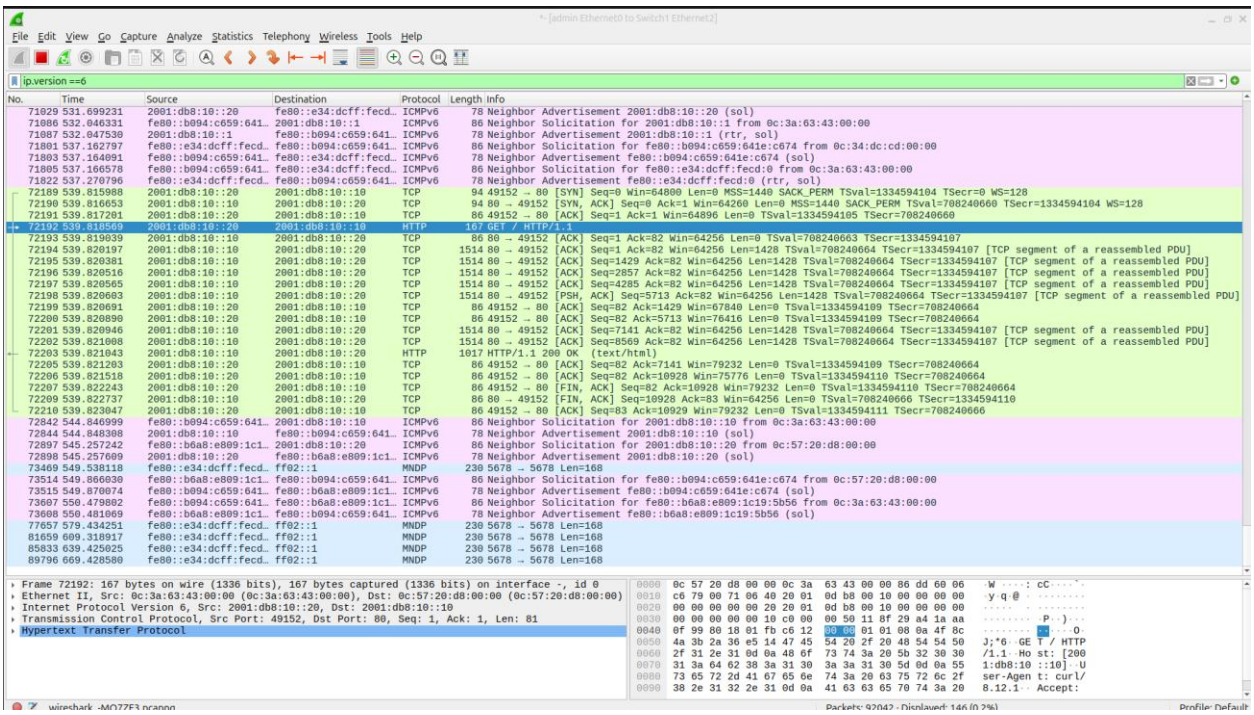
After completing the IPv6 configuration on the Admin PC, it is very important to verify that the entire network connectivity is correct. To add the Admin VM to the same subnet as the Web server, the IPv6 address has been successfully added using the sudo ip -6 addr add 2001:db8:10::20/64 dev ens3 command. To facilitate communication outside the network, the ether1 interface of the Router has been added as the default gateway using the sudo ip -6 route add default via 2001:db8:10::1 command. After the configuration is complete, it is very important to test the connectivity. So first, run ping6 -c 3 2001:db8:10::1 to verify that the router is responding. Then, check the Web server address with ping6 -c 3 2001:db8:10::10 to confirm that the Admin's communication with the Web server is active. The IPv6 address of the worker server was tested with ping6 -c 3 2001:db8:172::10, where the packet was routed and a successful reply was received [19].



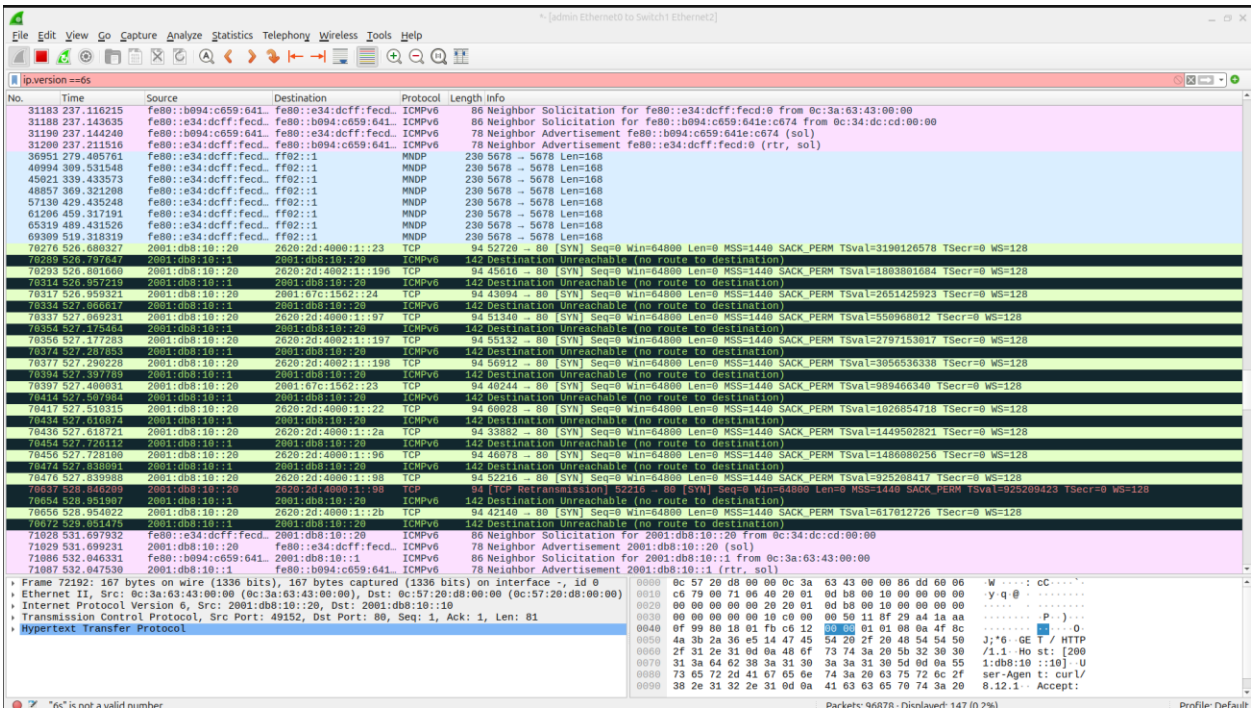


Then, the IPv6 (TCP/80) packet was captured on ens3 using Wireshark.

When the admin requests an HTTP page using IPv6 to the web server, the packets seen in Wireshark are different in several important ways compared to IPv4 traffic. The most obvious difference is that IPv6 first sees Neighbor Solicitation (NS) and Neighbor Advertisement (NA) packets, which act as ARP in IPv4. The first screenshot shows repeated NS/NA exchanges with the fe80:: link-local address. Through this, the admin has resolved the MAC address of its neighboring router and web server. Then, before the HTTP request is sent, a TCP handshake is performed over IPv6, which is seen on port 49152 to 80 in the capture. The large 128-bit IPv6 address is clearly visible in the packet header. In addition, IPv6 has a “Next Header” field (TCP = 6), which has replaced the IPv4 Protocol field [20].



After fixing the route, if we capture it again, the HTTP GET is properly converted to IPv6 and "GET / HTTP/1.1" is clearly visible in the payload.



Enabling IPv6 (on Host):

The steps are followed to enable IPv6 on the Host machine and make it work effectively. The complete process is explained step by step below -

- **Enable IPv6 on Host**

First, the commands are run to enable the IPv6 feature on the system, so that the Linux system enables the IPv6 protocol. net.ipv6.conf.all.disable_ipv6=0 means to enable IPv6 on the entire system without disabling it. net.ipv6.conf.default.disable_ipv6=0 means that IPv6 is enabled by default when a new interface is created. net.ipv6.conf.virbr0.disable_ipv6=0 indicates that IPv6 is also enabled on the virbr0 virtual bridge interface. After running these commands, IPv6 is temporarily enabled. To keep it permanently enabled, add the following three lines to the /etc/sysctl.conf file - net.ipv6.conf.all.disable_ipv6=0, net.ipv6.conf.default.disable_ipv6=0, net.ipv6.conf.virbr0.disable_ipv6=0. After adding these lines to the end of the file and saving, IPv6 remains enabled even after system reboot [18], [21].

- **Setting IPv6 address on virbr0 interface**

Next, the IPv6 address is added to the virbr0 interface, where the fd00::1/64 IPv6 address is added to the virbr0 interface. Here, fd00::/8 is the Unique Local Address (ULA) range, which is used in the local network, just like 192.168.x.x in IPv4. After checking, it can be seen that the IPv6 address has been successfully added to the virbr0 interface and is permanently active [22].

- **Testing IPv6 connectivity**

A two-way ping test is performed to test IPv6 connectivity, from the Router to the Host. This is done by sending a ping from the MikroTik router to the IPv6 address of the Host. Again, from the Host to the Router, which is done by sending a ping from the Host machine to the IPv6 address of the Router. Replies are received from both directions, indicating that the IPv6 network is fully functional and communication is occurring properly [23], [24].

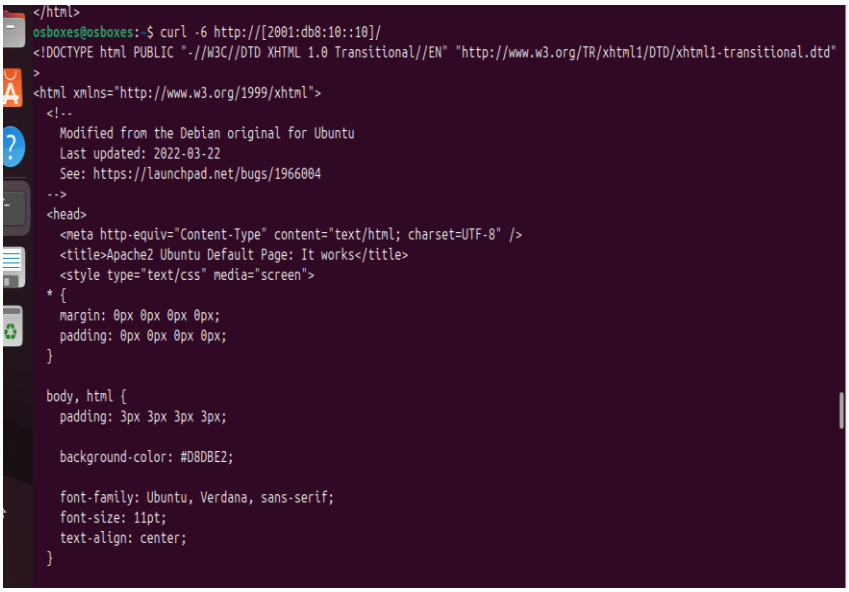
```
hasib@hasib-ThinkPad-T480s:~$ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=0
sudo sysctl -w net.ipv6.conf.default.disable_ipv6=0
sudo sysctl -w net.ipv6.conf.virbr0.disable_ipv6=0
net.ipv6.conf.all.disable_ipv6 = 0
net.ipv6.conf.default.disable_ipv6 = 0
net.ipv6.conf.virbr0.disable_ipv6 = 0
hasib@hasib-ThinkPad-T480s:~$ sudo nano /etc/sysctl.conf
hasib@hasib-ThinkPad-T480s:~$ sudo nano /etc/sysctl.conf
hasib@hasib-ThinkPad-T480s:~$ sudo ip -6 addr add fd00::1/64 dev virbr0
hasib@hasib-ThinkPad-T480s:~$ ip -6 addr show dev virbr0
4: virbr0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    inet6 fd00::1/64 scope global
       valid_lft forever preferred_lft forever
    inet6 fe80::5054:ff:fe93:1151/64 scope link
       valid_lft forever preferred_lft forever
hasib@hasib-ThinkPad-T480s:~$ ping6 fd00::2
PING fd00::2 (fd00::2) 56 data bytes
64 bytes from fd00::2: icmp_seq=1 ttl=64 time=1.78 ms
64 bytes from fd00::2: icmp_seq=2 ttl=64 time=1.57 ms
64 bytes from fd00::2: icmp_seq=3 ttl=64 time=1.37 ms
64 bytes from fd00::2: icmp_seq=4 ttl=64 time=1.50 ms
64 bytes from fd00::2: icmp_seq=5 ttl=64 time=1.48 ms
^C
--- fd00::2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4004ms
rtt min/avg/max/mdev = 1.365/1.538/1.776/0.135 ms
hasib@hasib-ThinkPad-T480s:~$
```

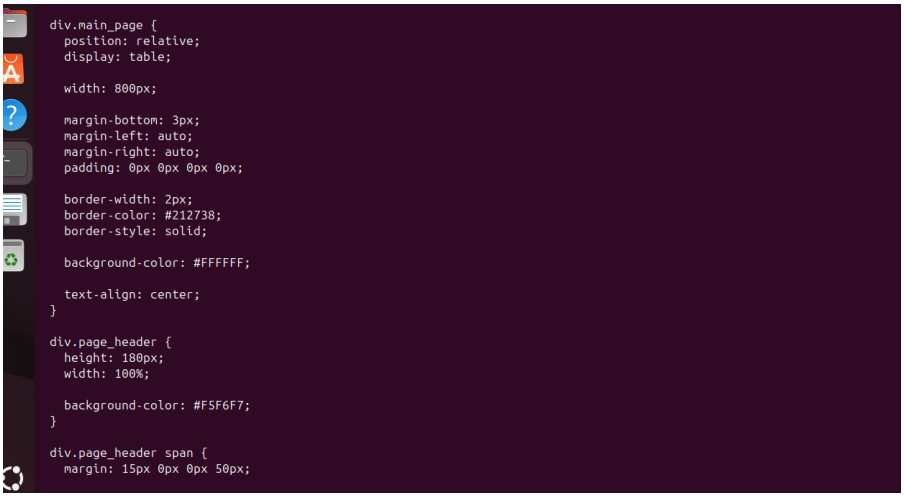
```
[admin@MikroTik] > /system package print
Columns: NAME, VERSION, BUILD-TIME, SIZE
# NAME      VERSION  BUILD-TIME           SIZE
0 routeros  7.16     2024-09-20 13:00:27  17.8MiB
[admin@MikroTik] > /ipv6 settings set disable-ipv6=no
[admin@MikroTik] > /ipv6 address add address=fd00::2/64 interface=ether4 advertise=yes
[admin@MikroTik] >
[admin@MikroTik] > /ipv6 address print
Flags: D - DYNAMIC; G - GLOBAL, L - LINK-LOCAL
Columns: ADDRESS, INTERFACE, ADVERTISE
#    ADDRESS                  INTERFACE  ADVERTISE
0 D  ::1/128                  lo         no
1 DL fe80::e34:dcff:fecd:3/64 ether4     no
2 DL fe80::e34:dcff:fecd:0/64 ether1     no
3 DL fe80::e34:dcff:fecd:2/64 ether3     no
4  G 2001:db8:10::1/64        ether1     no
5  G 2001:db8:172::1/64       ether3     no
6  G fd00::2/64               ether4     yes
[admin@MikroTik] > /ping fd00::1
  SEQ HOST                                     SIZE TTL TIME       STATUS
    0 fd00::1                                    56  64 2ms735us   echo reply
    1 fd00::1                                    56  64 1ms195us   echo reply
    2 fd00::1                                    56  64 1ms498us   echo reply
    3 fd00::1                                    56  64 1ms566us   echo reply
    4 fd00::1                                    56  64 1ms620us   echo reply
    5 fd00::1                                    56  64 1ms518us   echo reply
    6 fd00::1                                    56  64 1ms588us   echo reply
    sent=7 received=7 packet-loss=0% min-rtt=1ms195us avg-rtt=1ms674us
   max-rtt=2ms735us

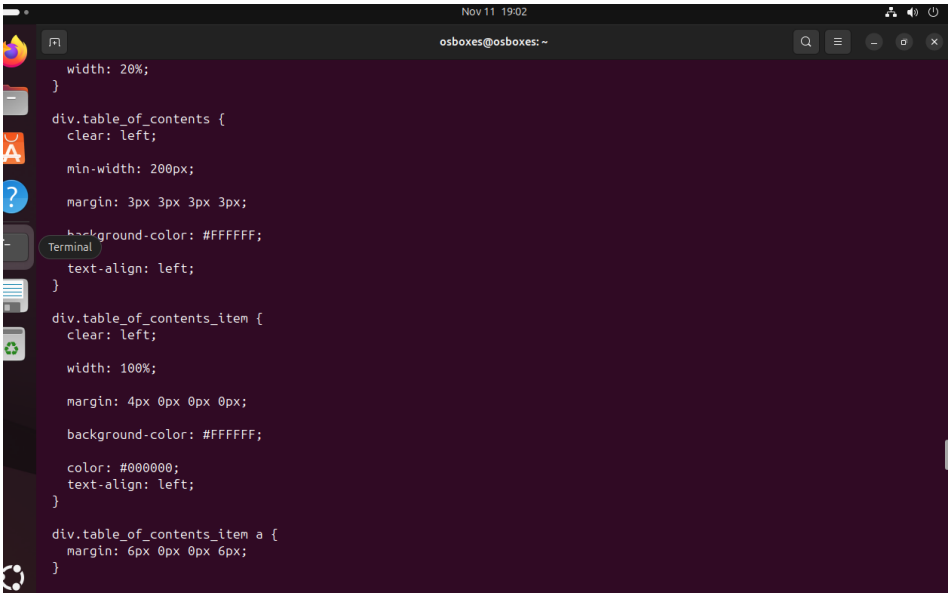
[admin@MikroTik] >
```

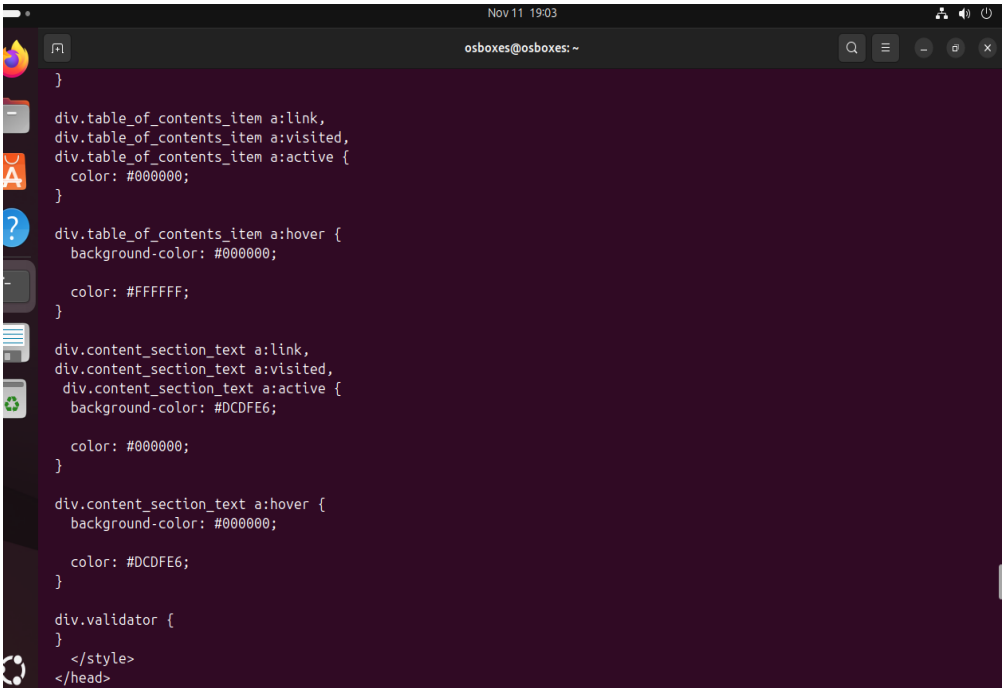
IPv6 HTTP Capture (Admin to Web):

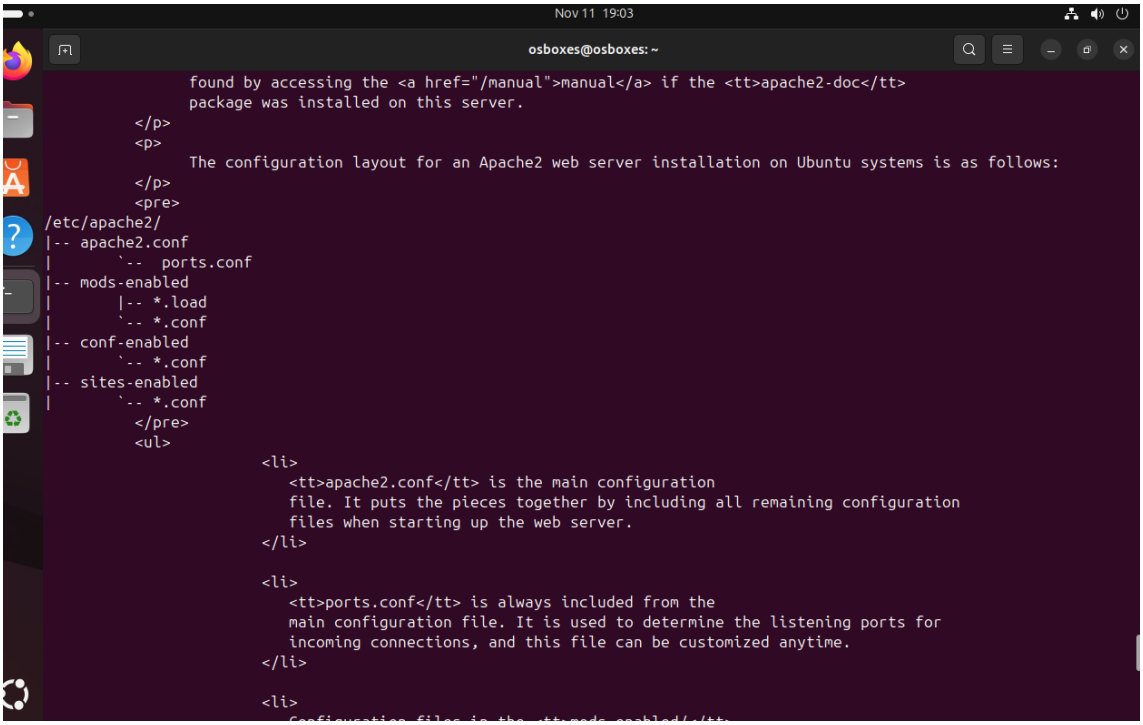
As before, capture was run on Router ether1 link. Using curl -6 from Admin, where the capture shows IPv6 packets (EtherType 0x86DD), large headers, ICMPv6 Neighbor Discovery, etc., not IPv4. The difference is IPv6 header (fixed 40 bytes), ND instead of ARP, etc. [23], [24].


osboxes@osboxes:~$ curl -6 http://[2001:db8:10::10]/
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"
>
<html xmlns="http://www.w3.org/1999/xhtml">
<!--
Modified from the Debian original for Ubuntu
Last updated: 2022-03-22
See: https://launchpad.net/bugs/1966004
-->
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Apache2 Ubuntu Default Page: It works</title>
<style type="text/css" media="screen">
* {
margin: 0px 0px 0px 0px;
padding: 0px 0px 0px 0px;
}
body, html {
padding: 3px 3px 3px 3px;
background-color: #D8DBE2;
font-family: Ubuntu, Verdana, sans-serif;
font-size: 11pt;
text-align: center;
}


div.main_page {
position: relative;
display: table;
width: 800px;
margin-bottom: 3px;
margin-left: auto;
margin-right: auto;
padding: 0px 0px 0px 0px;
border-width: 2px;
border-color: #212738;
border-style: solid;
background-color: #FFFFFF;
text-align: center;
}
div.page_header {
height: 180px;
width: 100%;
background-color: #F5F6F7;
}
div.page_header span {
margin: 15px 0px 0px 50px;


Nov 11 19:02
osboxes@osboxes: ~
Terminal
width: 20%;
}
div.table_of_contents {
clear: left;
min-width: 200px;
margin: 3px 3px 3px 3px;
background-color: #FFFFFF;
text-align: left;
}
div.table_of_contents_item {
clear: left;
width: 100%;
margin: 4px 0px 0px 0px;
background-color: #FFFFFF;
color: #000000;
text-align: left;
}
div.table_of_contents_item a {
margin: 6px 0px 0px 6px;
}


Nov 11 19:03
osboxes@osboxes: ~
}
div.table_of_contents_item a:link,
div.table_of_contents_item a:visited,
div.table_of_contents_item a:active {
color: #000000;
}
div.table_of_contents_item a:hover {
background-color: #000000;
color: #FFFFFF;
}
div.content_section_text a:link,
div.content_section_text a:visited,
div.content_section_text a:active {
background-color: #DCDFE6;
color: #000000;
}
div.content_section_text a:hover {
background-color: #000000;
color: #DCDFE6;
}
div.validator {
}
</style>
</head>

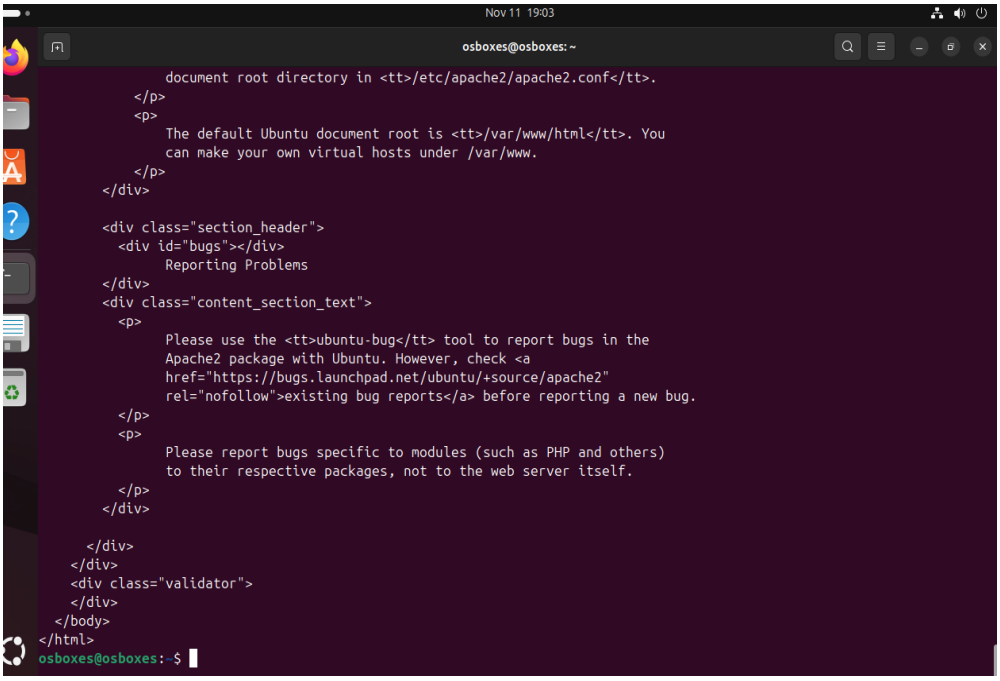

Nov 11 19:03
osboxes@osboxes: ~
found by accessing the manual if the <tt>apache2-doc</tt>
package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

<pre>
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf
</pre>

<tt>apache2.conf</tt> is the main configuration
file. It puts the pieces together by including all remaining configuration
files when starting up the web server.

<tt>ports.conf</tt> is always included from the
main configuration file. It is used to determine the listening ports for
incoming connections, and this file can be customized anytime.



Address compress/decompress:

From host PC

This step completes the IPv6 Address Compress and Decompress (Expand) process, where a full IPv6 address is compressed and then expanded back to its full form [22].

Compressing IPv6 Address (Compact form)

First, a complete IPv6 address is taken, which contains many zeros (0), which is convenient to condense in IPv6 notation. It is converted to a short or compact format using Python's ipaddress module.

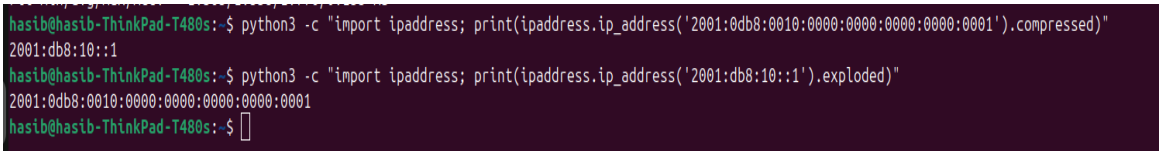
The Python ipaddress library automatically compresses the zeros in the IPv6 address. 2001:db8:10::1 is the compact form of the IPv6 address.

Here, the :: symbol is used in place of consecutive blocks of zeros, so the entire address can be expressed much shorter and more simply [22], [25].

Expanding the Compact IPv6 Address (Full form)

Now, conversely, the compact IPv6 address is converted back to its full or complete form. This converts the previous compact IPv6 address back to its full form. The output is: 2001:0db8:0010:0000:0000:0000:0000:0001

This is the Full (Expanded) form of the IPv6 address, where each block is represented by a 4-digit hexadecimal value and no leading zeros are omitted [22].



For Web PC

First, a complete IPv6 address is used: 2001:0db8:0010:0000:0000:0000:0000:0010

This address contains multiple consecutive zero blocks (0000), which are compressed into IPv6 notation to appear in a shorter form.

The following Python command is run for this conversion, where the ipaddress module automatically converts the IPv6 address to a compact or short format.

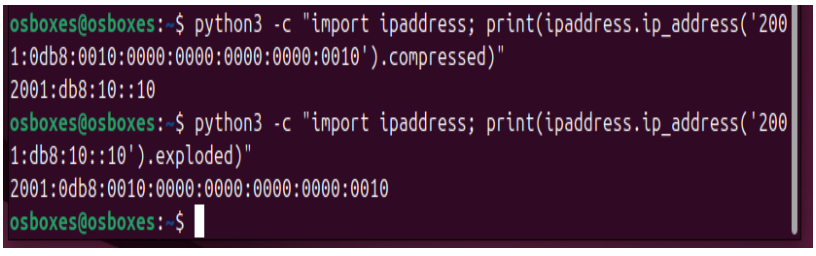
Output: 2001:db8:10::10

Here, the :: symbol is used instead of consecutive zero blocks. :: can only be used once in an IPv6 address, and it represents the number of zero blocks. As a result, the address is much shorter and easier to write [25], [26].

Expanding the Compact IPv6 Address in Next

The previously compact address is now converted back to its full form using the following command, which re-displays each block of the IPv6 address as a four-digit (hexadecimal) number and fills in the hidden zeros [25].

Output: 2001:0db8:0010:0000:0000:0000:0000:0010



For Worker PC:

Output: 2001:0db8:0172:0000:0000:0000:0000:0010

```
osboxes@osboxes:~$ python3 -c "import ipaddress; print(ipaddress.ip_address('200
1:db8:172::10').exploded)"
2001:0db8:0172:0000:0000:0000:0000:0010
osboxes@osboxes:~$
```

Viewing IPv4 and IPv6 addresses together on an Ubuntu host machine:

echo "[IPv4]" This prints the [IPv4] header. This indicates a list of IPv4 addresses in the output. ip -4 -o addr show scope global This section shows IPv4 addresses using the Linux ip command. Here -4 means to show only IPv4 addresses. -o (one-line output) means to display information for each interface on one line, which is convenient for scripting. scope global means to show only global (i.e. active/available on the network) addresses, not local or loopback. This output is then formatted with awk as awk '{print " "$2": "$4}'. Here $2 is the interface name (e.g. ens3, virbr0) and $4 is the IPv4 address of that interface (e.g. 192.168.10.5/24) [13], [27].

echo "[IPv6]" This displays the [IPv6] header, indicating that IPv6 addresses are now displayed.

ip -6 -o addr show scope global | awk '{print " "$2": "$4}' This works in the same way as displaying IPv6 addresses. The -6 means only IPv6 addresses are displayed. The scope global only shows addresses that are validly in use on the network (excluding link-local addresses). The awk format nicely displays the name and IPv6 address of each interface [13], [27].

```
hasib@hasib-ThinkPad-T480s:~$ echo "[IPv4]"
ip -4 -o addr show scope global | awk '{print "  "$2": "$4}'
echo "[IPv6]"
ip -6 -o addr show scope global | awk '{print "  "$2": "$4}'
[IPv4]
  wlp61s0: 10.91.59.145/20
  virbr0: 192.168.122.1/24
  docker0: 172.17.0.1/16
[IPv6]
  virbr0: fd00::1/64
hasib@hasib-ThinkPad-T480s:~$
```

For Router (MikroTik CHR):

In the Router OS terminal:

The following commands are used to view IPv4 and IPv6 address configurations in the MikroTik RouterOS terminal.

/ip address print This command displays information about all IPv4 addresses and interfaces of the router.

- ADDRESS: The IPv4 address and subnet mask of the router.
- NETWORK: The network ID of that address.
- INTERFACE: Shows which interface (such as ether1 or ether3) the address is assigned to.
- Flags: Specifies whether the address is enabled, disabled, or dynamic (obtained from DHCP or another source).

/ipv6 address print detail This command displays detailed information about all IPv6 addresses configured on the router [13], [26].

- **ADDRESS:** The IPv6 address of the router.
- **FROM-POOL:** If the address is taken from an IPv6 pool, it will be displayed here (if not, it will be blank).
- **INTERFACE:** The interface to which the IPv6 address is assigned (e.g. ether1 or ether3).
- **ADVERTISE:** Indicates whether the router will send IPv6 Router Advertisements (yes or no).

• **Flags:**

  - **G** = Global address (suitable for use on the network)
  - **L** = Link-local (automatically generated local address)
  - **D** = Dynamic (obtained from DHCPv6 or SLAAC)
  - **X** = Disabled (disabled)

```
[admin@MikroTik] > /ipv6 address print detail
Flags: X - disabled, I - invalid, D - dynamic; G - global, L - link-local;
S - slave; d - deprecated
 0 D    address=::1/128 from-pool="" interface=lo actual-interface=lo eui-64=no
        advertise=no no-dad=no

 1 DL   address=fe80::e34:dcff:fecd:3/64 from-pool="" interface=ether4
        actual-interface=ether4 eui-64=no advertise=no no-dad=no

 2 DL   address=fe80::e34:dcff:fecd:0/64 from-pool="" interface=ether1
        actual-interface=ether1 eui-64=no advertise=no no-dad=no

 3 DL   address=fe80::e34:dcff:fecd:2/64 from-pool="" interface=ether3
        actual-interface=ether3 eui-64=no advertise=no no-dad=no

 4  G   address=2001:db8:10::1/64 from-pool="" interface=ether1
        actual-interface=ether1 eui-64=no advertise=no no-dad=no

 5  G   address=2001:db8:172::1/64 from-pool="" interface=ether3
        actual-interface=ether3 eui-64=no advertise=no no-dad=no

 6  G   address=fd00::2/64 from-pool="" interface=ether4
        actual-interface=ether4 eui-64=no advertise=yes no-dad=no
[admin@MikroTik] >
```

For Full/Compact conversion, go to Ubuntu Host:

```
hasib@hasib-ThinkPad-T480s:~$ sudo apt install -y sipcalc
sipcalc 2001:db8:10::1
[sudo] password for hasib:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
sipcalc is already the newest version (1.1.6-3).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 134 not upgraded.
-[ipv6 : 2001:db8:10::1] - 0

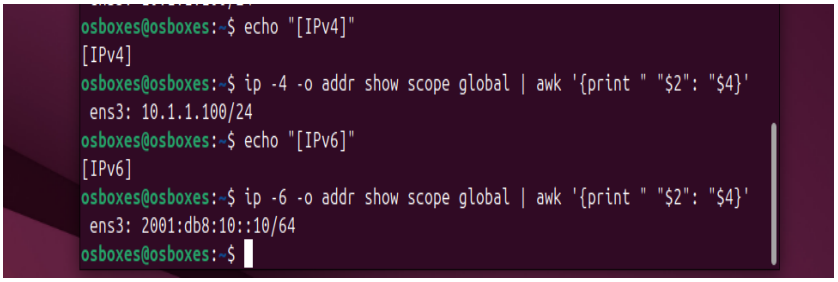
[IPV6 INFO]
Expanded Address        - 2001:0db8:0010:0000:0000:0000:0000:0001
Compressed address      - 2001:db8:10::1
Subnet prefix (masked)  - 2001:db8:10:0:0:0:0:1/128
Address ID (masked)     - 0:0:0:0:0:0:0:0/128
Prefix address          - ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff
Prefix length           - 128
Address type            - Aggregatable Global Unicast Addresses
Network range           - 2001:0db8:0010:0000:0000:0000:0000:0001 -
                          2001:0db8:0010:0000:0000:0000:0000:0001

-
hasib@hasib-ThinkPad-T480s:~$
```

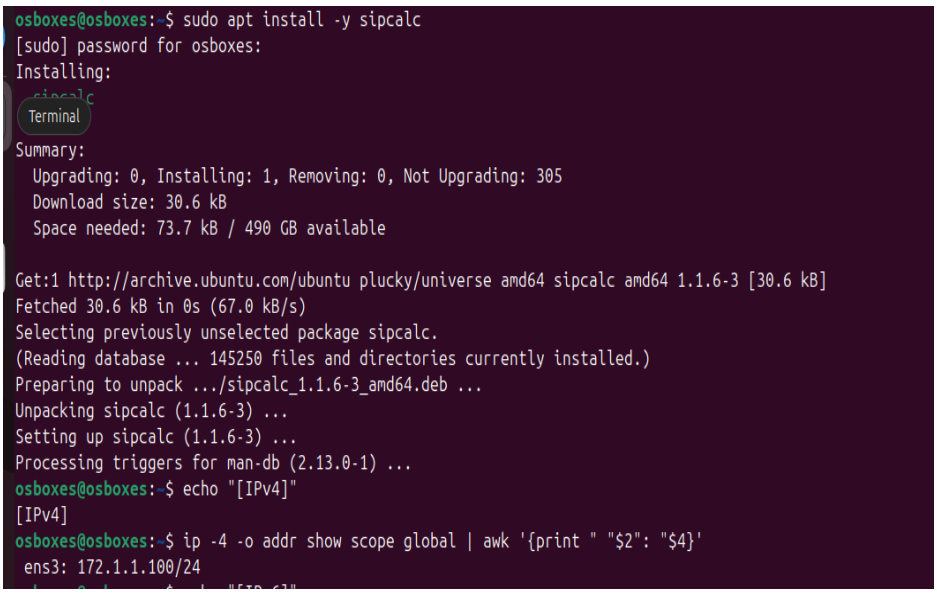
For **Admin PC (Ubuntu):**

```
osboxes@osboxes:~$ echo "[IPv4]"
[IPv4]
osboxes@osboxes:~$ ip -4 -o addr show scope global | awk '{print" "$2": "$4}'
 ens3: 10.1.1.99/24
osboxes@osboxes:~$ echo "[IPv6]"
[IPv6]
osboxes@osboxes:~$ ip -6 -o addr show scope global | awk '{print" "$2": "$4}'
 ens3: 2001:db8:10::20/64
osboxes@osboxes:~$
```

For Web PC:



For Worker PC:



Ping through the Router:

I did ping the router at different websites like: googe.com, youtube.com (where packet-loss was 0%)

```
[admin@MikroTik] > /ping 8.8.8.8
  SEQ HOST                                     SIZE TTL TIME       STATUS
    0 8.8.8.8                                    56 111 45ms778us
    1 8.8.8.8                                    56 111 46ms344us
    2 8.8.8.8                                    56 111 45ms966us
    3 8.8.8.8                                    56 111 46ms724us
    4 8.8.8.8                                    56 111 46ms402us
    5 8.8.8.8                                    56 111 45ms313us
    6 8.8.8.8                                    56 111 46ms588us
    7 8.8.8.8                                    56 111 46ms213us
    8 8.8.8.8                                    56 111 47ms65us
    sent=9 received=9 packet-loss=0% min-rtt=45ms313us avg-rtt=46ms265us
    max-rtt=47ms65us

[admin@MikroTik] > /ping google.com
  SEQ HOST                                     SIZE TTL TIME       STATUS
    0 216.239.38.120                             56 110 40ms876us
    1 216.239.38.120                             56 110 41ms118us
    2 216.239.38.120                             56 110 40ms724us
    3 216.239.38.120                             56 110 42ms742us
    4 216.239.38.120                             56 110 41ms349us
    5 216.239.38.120                             56 110 41ms28us
    6 216.239.38.120                             56 110 41ms224us
    7 216.239.38.120                             56 110 40ms520us
    8 216.239.38.120                             56 110 41ms168us
    sent=9 received=9 packet-loss=0% min-rtt=40ms520us avg-rtt=41ms194us
    max-rtt=42ms742us

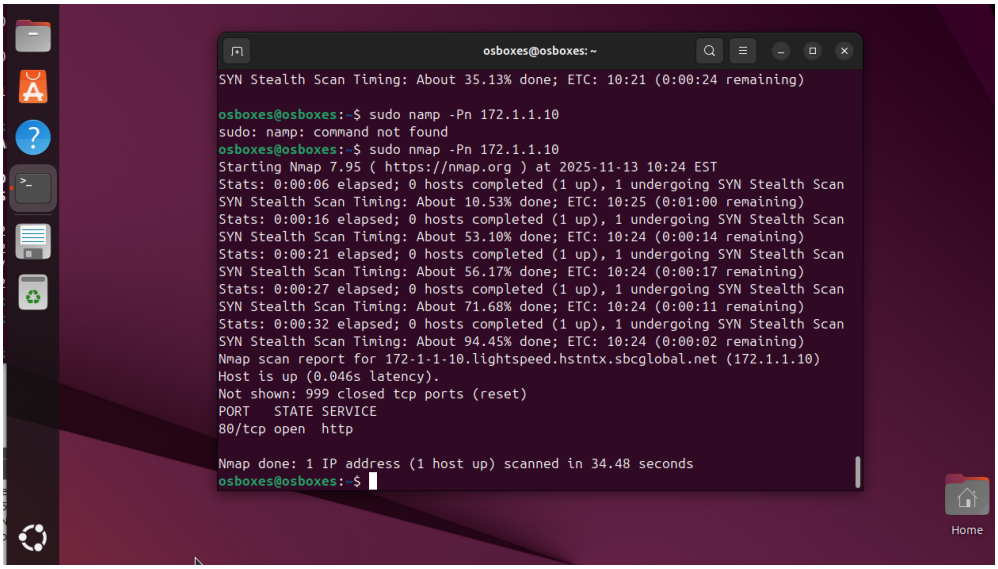
[admin@MikroTik] > /ping youtube.com
  SEQ HOST                                     SIZE TTL TIME       STATUS
    0 64.233.164.91                              56 108 43ms611us
    1 64.233.164.91                              56 108 43ms188us
    2 64.233.164.91                              56 108 43ms962us
    3 64.233.164.91                              56 108 40ms541us
    4 64.233.164.91                              56 108 40ms765us
    sent=5 received=5 packet-loss=0% min-rtt=40ms541us avg-rtt=42ms413us
    max-rtt=43ms962us

[admin@MikroTik] >
```

Task 1 Part 4 Solutions:

4.1 - Scanning the Worker VM from Admin:

Used to scan Worker (172.1.1.10) from Admin PC:

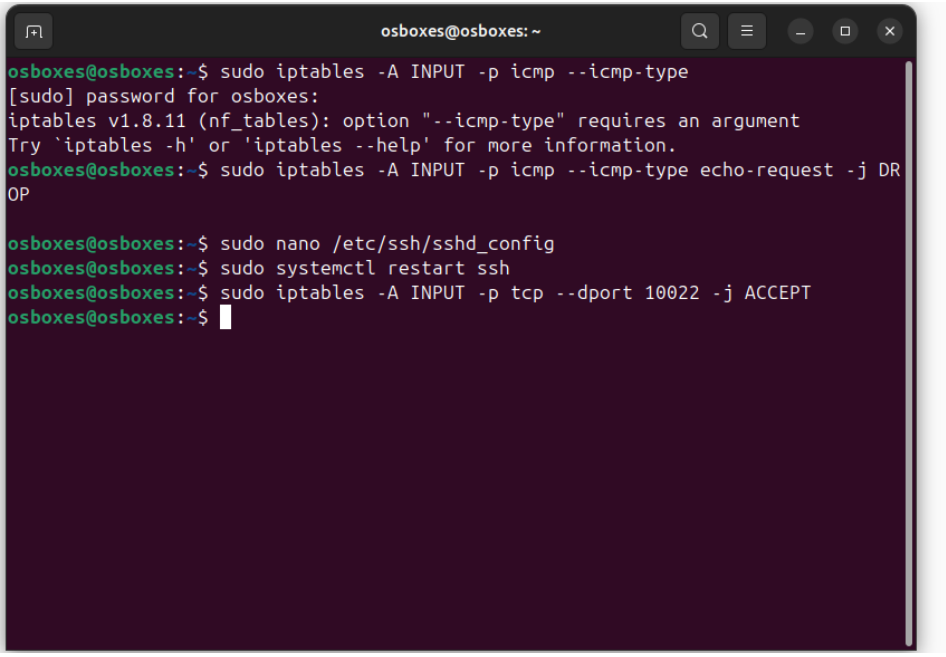


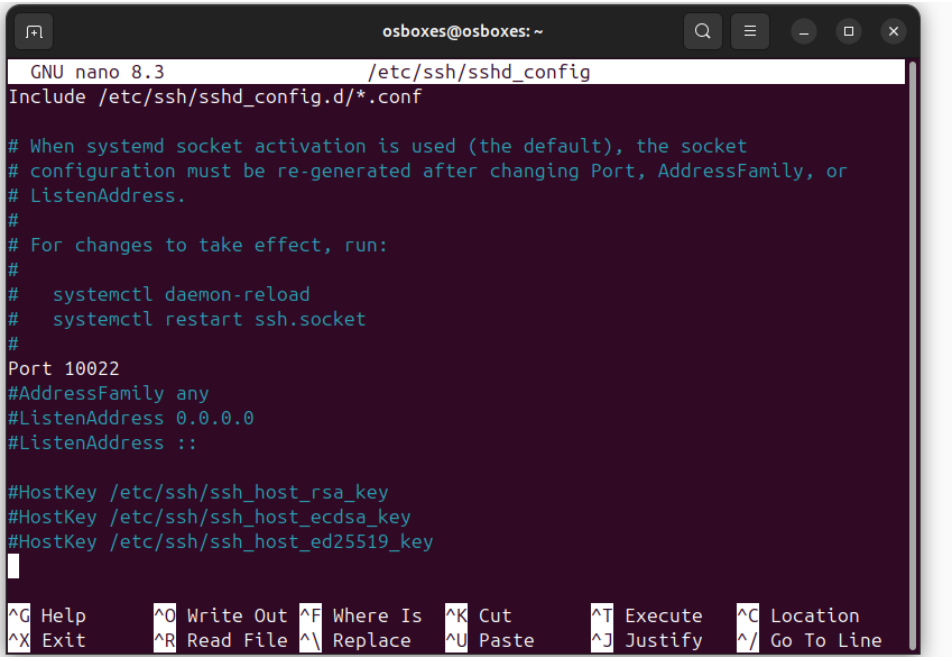
After scanning, it was found that Worker only showed 80/tcp open and almost all other ports were

closed or filtered. The default configuration of Worker only had Apache (port 80) running - SSH (port 22) or any other service was not running, so Nmap only detected port 80 open.

4.2 - Block ICMP on Worker & Change SSH port to a port above 10000

In this section, two security-related configurations are made on the Worker machine - the first is to block ICMP and the second is to change the SSH port. In the first step, ICMP echo-requests are prohibited on the Worker PC, which means that the Worker will no longer accept any incoming ping requests. This is useful for security, because ICMP pings are usually used to determine whether the host is alive, and blocking these requests prevents external sources from pinging the Worker. Therefore, iptables was used to block IPv4, and ip6tables was used to block IPv6. After applying these rules, when the Admin pings the Worker, no response is received and “packet loss” or “Host unreachable” is displayed, indicating that ICMP has been successfully blocked [28]. In the second step, the default port of the SSH server on the Worker machine is changed from 22 to 10022, which is a common way to protect the system from brute-force attacks. The service is restarted by changing the port number in the SSH configuration file so that the new port takes effect. The new SSH port is then allowed in the firewall, allowing SSH connections to the Worker from the outside. This means that SSH is now running on port 10022 instead of the default port 22, which increases security and reduces unauthorized scanning. The whole process strengthens access control on the Worker machine and improves system security [28].





4.3 - Scanning it again

Normal scan again from Admin:

After running sudo nmap 172.1.1.10 from Admin, Nmap shows that the Worker host appears to be “down”. In fact, the Worker host is up, but Nmap is unable to detect it. The main reason for this is the previous security configurations made on the Worker. First, ICMP echo-requests are blocked on the Worker system, which means that Nmap does not receive any response when pinging it in its default host discovery phase. When Nmap does not receive an ICMP replay, it assumes that the host is offline. Second, the default port 22 for SSH has been changed to 10022, so when Nmap scans port 22 normally, it shows it as closed or filtered. This makes Nmap more certain that the host is “down” or inaccessible by not finding any known services open on the default port.

In this case, the Nmap scan output shows - “Host seems down. If it is really up, but blocking our ping probes, try -Pn” - which directly indicates that the normal discovery step of Nmap has failed due to the Worker host blocking ICMP. That is, Nmap cannot see the host, because the ping is not going and there is no response on the default scanned ports. So it shows 0 hosts up, although in reality the host is up, because ICMP is prohibited for security purposes on the Worker and SSH port changes are enforced.

```
osboxes@osboxes:~$ sudo nmap 172.1.1.10
[sudo] password for osboxes:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-13 11:17 EST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.11 seconds
```

4.4 - Force Nmap to scan all ports even if ICMP is blocked

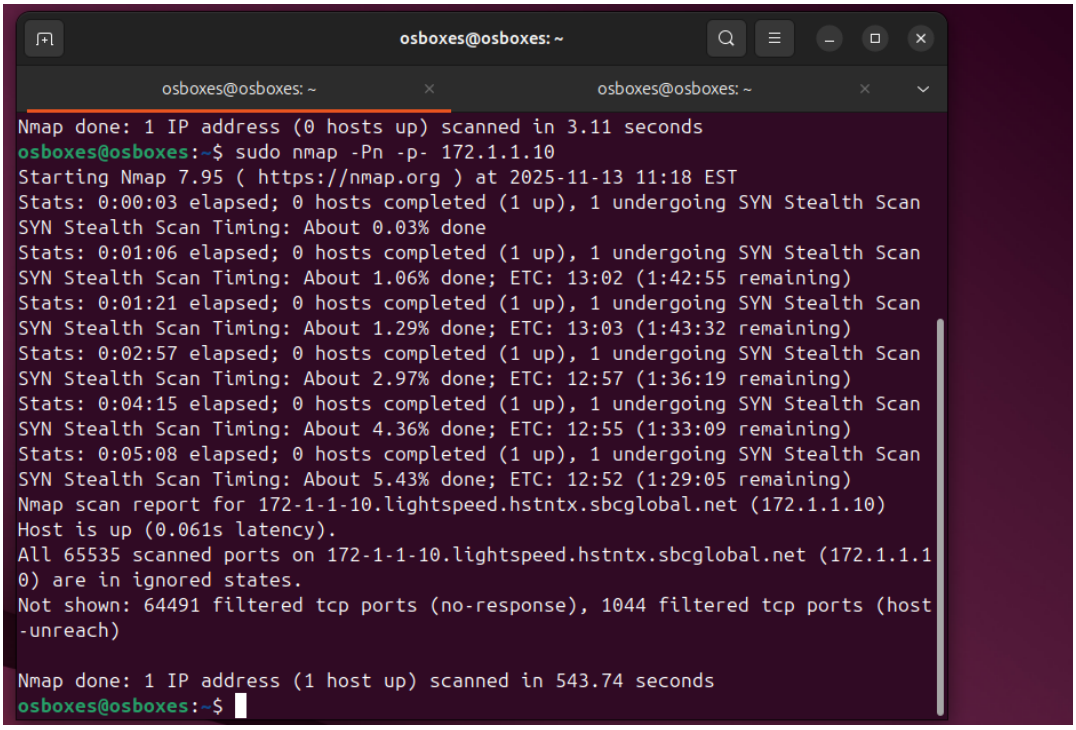
If Nmap does not receive a normal ping, it considers the target dead.

So, you need to force scan:

-Pn : skip ping and scan (ICMP block bypass)

-p- : scan all ports from 1 to 65535

After running sudo nmap 172.1.1.10 from admin, Nmap shows that the Worker host appears to be “down”. In fact, the Worker host is up, but Nmap is unable to detect it. The main reason for this is the previous security configurations made on the Worker. First, ICMP echo-request is blocked on the Worker system, as a result of which Nmap does not receive any response by sending a ping in its default host discovery step. When Nmap does not receive an ICMP replay, it assumes that the host is offline. Second, the default port 22 of SSH has been changed to 10022, so when Nmap scans port 22 normally, it shows it closed or filtered. As a result, Nmap is more certain that the host is “down” or inaccessible by not finding any known services open on the default port. In this case, the Nmap scan output shows -“Host seems down. If it is really up, but blocking our ping probes, try -Pn”- which directly indicates that the normal discovery step of Nmap has failed due to the Worker host blocking ICMP. That is, Nmap cannot see the host, because the ping is not going through and there is no response on the default scanned ports. So it shows 0 hosts up, even though the host is actually up.

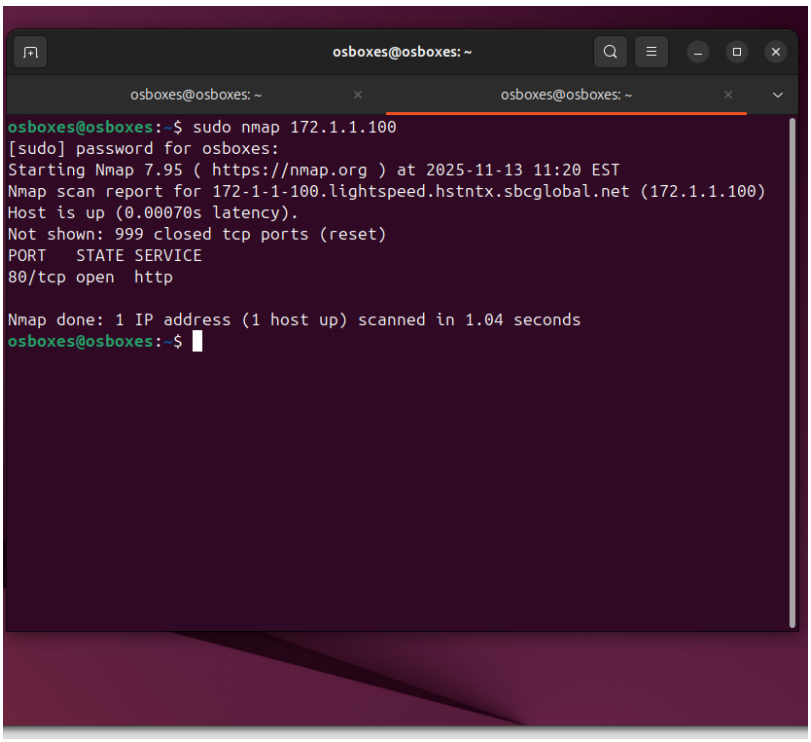


4.5 : Gather some information about open ports on Web (SSH + HTTP)

Both SSH and HTTP were open on the Web server (172.1.1.100):

Nmap Service Detection:

In this step, a simple Nmap scan was run from the Admin machine to detect the open ports on the Web server. The IP of the Web server was 172.1.1.100, and it was previously known that SSH and HTTP services were running there. However, after running the scan, it was seen that Nmap was only able to detect HTTP port 80 as open. The scan output showed "Host is up", which indicates that the server is up and responding. Nmap then reported that 999 TCP ports were reset, and the only open port was 80/tcp, which is used for HTTP services.



References:

[1] GNS3, "Architecture," *GNS3 Documentation*. Available: https://docs.gns3.com/docs/using-gns3/design/architecture/

[2] "What is Port 22: A Complete Guide for Beginners," *SSL Insights*, May 22, 2025. Available: https://sslinsights.com/what-is-port-22/ SSL Insights

[3] S. M. Tabish, “How to Use Nmap to Scan for Open Ports,” *phoenixNAP Knowledge Base*, Oct. 31, 2024. Available: https://phoenixnap.com/kb/nmap-scan-open-ports/

[4] Nmap Project, “Host Discovery,” *Nmap Documentation*. Available: https://nmap.org/docs/discovery.

[5] T. H. B. (Red Hat), “An introduction to using tcpdump at the Linux command line,” *Red Hat Blog*. Available: https://www.redhat.com/en/blog/introduction-using-tcpdump-linux-command-line.

[6] Wireshark Wiki, “SSH,” *Wireshark Wiki*. Available: https://wiki.wireshark.org/SSH.

[7] MikroTik Documentation, “/ipv6 settings - IPv6 Settings Sub-menu,” *RouterOS Documentation*. Available: https://help.mikrotik.com/docs/spaces/ROS/pages/103841817/IP%2BSettings.

[8] DED9, “How to Add IPv6 to MikroTik: A Step-by-Step Guide,” blog article. Available: https://ded9.com/mikrotik-adding-ipv6-tutorial/.

[9] Major Hayden, “Native IPv6 connectivity in MikroTik’s RouterOS,” *major.io* blog. Available: https://major.io/p/native-ipv6-connectivity-in-mikrotiks-routeros/.

[10] mitmproxy Project, “Certificates mitmproxy documentation,” *mitmproxy Docs*. Available: https://docs.mitmproxy.org/stable/concepts/certificates/.

[11] mitmproxy Community Discussion, “Downloading mitmproxy CA certificate from mitm.it,” *mitmproxy GitHub Discussions*. Available: https://github.com/mitmproxy/mitmproxy/discussions/7014

[12] libvirt Project, “Virtual Networking (virbr0 default network),” *libvirt Wiki*. Available: https://wiki.libvirt.org/VirtualNetworking.html.

[13] T. Eggert et al., “ip-address(8) - Linux manual page,” *man7.org*. Available: https://www.man7.org/linux/man-pages/man8/ip-address.8.html.

[14] “ip (8) - Linux man page,” *linux.die.net*. Available: https://linux.die.net/man/8/ip.

[15] Netplan Developers, “Network configuration examples - Adding default IPv6 route,” *Netplan.io Docs*. Available: https://netplan.io/examples

[16] **MikroTik Wiki,** “Manual: IPv6,” *MikroTik Wiki*. Available: https://wiki.mikrotik.com/wiki/Manual:IPv6.

[17] “ip-route(8) - Linux Routing Commands,” *man7.org*. Available: https://www.man7.org/linux/man-pages/man8/ip-route.8.html

[18] Red Hat, “Configuring IP networking with ip commands,” *Red Hat Documentation*. Available:

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/networking_guide/sec-configuring_ip_networking_with_ip_commands

[19] MikroTik Documentation, “Torch - RouterOS Traffic Monitoring Tool,” *RouterOS Docs*. Available: https://help.mikrotik.com/docs/spaces/ROS/pages/8323150/Torch.

[20] MikroTik Documentation, “Packet Sniffer - RouterOS,” *RouterOS Docs*. Available: https://help.mikrotik.com/docs/spaces/ROS/pages/8323088/Packet%2BSniffer.

[21] Linux Kernel Docs, “IP sysctl - IPv6 parameters (disable_ipv6),” *kernel.org Documentation*. [Online]. Available: https://www.kernel.org/doc/html/latest/networking/ip-sysctl.html

[22] R. Hinden and B. Haberman, *“Unique Local IPv6 Unicast Addresses (RFC 4193),”* IETF. Available: https://www.rfc-editor.org/rfc/rfc4193

[23] MikroTik Documentation, “Ping Tool (IPv4/IPv6 ICMP),” *RouterOS Docs*. Available: https://help.mikrotik.com/docs/display/ROS/Ping

[24] **Linux Man Pages**, “ping(8) - IPv6 with -6 flag,” *man7.org*. Available: https://www.man7.org/linux/man-pages/man8/ping.8.html

[25] Python Software Foundation, “ipaddress - IPv4/IPv6 manipulation library,” *Python 3 Standard Library Documentation*. Available: https://docs.python.org/3/library/ipaddress.html

[26] S. Deering and R. Hinden, “Internet Protocol Version 6 (IPv6) Addressing Architecture,” *IETF RFC 4291*, 2006. Available: https://www.rfc-editor.org/rfc/rfc4291

[27] Linux Foundation, “iproute2 - ip command overview,” *Linux Foundation Documentation*. Available: https://wiki.linuxfoundation.org/networking/iproute2

[28] Red Hat. *5.11. Managing ICMP Requests | Security Guide - Red Hat Enterprise Linux 7*. Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/security_guide/sec-managing_icmp_requests.